WEB PROGRAMMING

WEEKS 5, 7, 8, 9, 10, 11 & 12

Every program has a clear title. React programs are grouped by project, and each source file remains in its original program section.

PROGRAM INDEX

WEEK 5

Code 1 — Paragraph Highlight (HTML + JavaScript)
Code 2 — User Information Form (HTML + JavaScript)
Code 3 — Layered Image Showcase (HTML + JavaScript)
Code 4 — Live Textarea Editor (HTML + JavaScript)
Code 5 — Kids' Sketch Pad (HTML + JavaScript)

WEEK 7

Code 1 — Student Attendance Tracker
Code 2 — Movie Genre Poll
Code 3 — Student Quiz Evaluation System
Code 4 — Shopping Bill Calculator
Code 5 — Fitness Activity Planner

WEEK 8

Code 1 — Employee Directory (React)
Code 2 — Stopwatch (React)
Code 3 — Kids' Sketch Pad
Code 4 — Layered Image Showcase
Code 5 — Live Textarea Editor

WEEK 9

Code 1 — Medical Shop Inventory (React)
Code 2 — Food Order Management (React)
Code 3 — Music Store Inventory (React)
Code 4 — Bookstore Inventory (React)

WEEK 10

Code 1 — Employee Directory (React Class Component)
Code 2 — Stopwatch (React Class Component)
Code 3 — Crop Dashboard (React)
Code 4 — Task Management Dashboard (React)

WEEK 11

Code 1 — Book Card (React Props and PropTypes)

Code 2 — Price Tag with Discount (React State)
Code 3 — Student Dashboard and Marks (React Router)
Code 4 — Employee Card (React Props)
Code 5 — Recipe Card with Favourite Toggle (React Hooks)

WEEK 12

Code 1 — Transit Connect Metro Departure Board
Code 2 — Weather Point Temperature Converter
Code 3 — NoteCraft Task Board
Code 4 — BrewNova Coffee Menu
Code 5 — Recipe Search and Filter

WEEK 5 — PREVIOUSLY PROVIDED CODES

Code 1 — Paragraph Highlight (HTML + JavaScript)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Highlight Paragraph</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
    <style>
        .highlight {
            background-color: yellow;
        }

        #textPara {
            padding: 5px;
            text-align: center;
        }
    </style>
</head>
<body>
    <div class="container mt-5">
        <p id="textPara">This is a sample paragraph to be highlighted.</p>
        <div class="text-center">
            <button id="highlightBtn" type="button" class="btn btn-primary">Highlight</button>
        </div>
    </div>
    <script>
        document.addEventListener("DOMContentLoaded", function () {
            const textPara = document.getElementById("textPara");
            const highlightBtn = document.getElementById("highlightBtn");
            highlightBtn.addEventListener("click", function () {
                textPara.classList.toggle("highlight");
            });
        });
    </script>
</body>
</html>
```

Code 2 — User Information Form (HTML + JavaScript)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>User Information Form</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
    <div class="container mt-5">
        <h1>User Information Form</h1>
        <form id="userForm" novalidate>
            <div class="mb-3">
                <label for="name" class="form-label">Name</label>
                <input type="text" id="name" class="form-control">
                <div id="nameError" class="invalid-feedback">Please enter your name.</div>
            </div>
            <div class="mb-3">
                <label for="email" class="form-label">Email</label>
                <input type="text" id="email" class="form-control">
                <div id="emailError" class="invalid-feedback">Please enter a valid email address ending with .com.</div>
            </div>
            <div class="mb-3">
                <label for="age" class="form-label">Age</label>
                <input type="number" id="age" class="form-control">
                <div id="ageError" class="invalid-feedback">Age must be between 18 and 120.</div>
            </div>
            <button type="submit" id="submitBtn" class="btn btn-primary">Submit</button>
        </form>
        <div id="successMessage" class="alert alert-success mt-3 d-none">
            <div id="successData"></div>
        </div>
        <div id="currentColor"></div>
    </div>
    <script>
        document.addEventListener("DOMContentLoaded", function () {
            const form = document.getElementById("userForm");
            const name = document.getElementById("name");
            const email = document.getElementById("email");
            const age = document.getElementById("age");
            const nameError = document.getElementById("nameError");
            const emailError = document.getElementById("emailError");
            const ageError = document.getElementById("ageError");
            const successMessage = document.getElementById("successMessage");
            const successData = document.getElementById("successData");
            form.addEventListener("submit", function (event) {
                event.preventDefault();
                nameError.style.display = "none";
                emailError.style.display = "none";
                ageError.style.display = "none";
                name.classList.remove("is-invalid");
                email.classList.remove("is-invalid");
                age.classList.remove("is-invalid");
                successMessage.classList.add("d-none");
                const n = name.value.trim();
                const e = email.value.trim();
                const a = age.value.trim();
                let valid = true;
                if (n === "") {
                    nameError.style.display = "block";
                    name.classList.add("is-invalid");
                    valid = false;
                }
```

```
            if (!/^[^\s@]+@[^\s@]+\.com$/i.test(e)) {
                emailError.style.display = "block";
                email.classList.add("is-invalid");
                valid = false;
            }
            if (a === "" || Number(a) < 18 || Number(a) > 120) {
                ageError.style.display = "block";
                age.classList.add("is-invalid");
                valid = false;
            }
            if (!valid) return;
            successData.textContent = "Success! Here is the data: Name: " + n + " | Email: " + e + " | Age: " + a;
            successMessage.classList.remove("d-none");
            form.reset();
        });
    });
    </script>
</body>
</html>
```

Code 3 — Layered Image Showcase (HTML + JavaScript)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Layered Image Showcase</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
    <style>
        .showcase { text-align: center; margin: 30px auto 60px; }
        .image-container { position: relative; width: 300px; height: 300px; margin: 20px auto; }
        .layered-img { position: absolute; width: 180px; height: 240px; object-fit: cover; left: 50%; top: 50%; transform: translate(-50%, -50%);
border: 1px solid #ddd; }
        #img1 { z-index: 1; }
        #img2 { z-index: 2; }
        #img3 { z-index: 3; }
    </style>
</head>
<body>
    <section class="showcase">
        <h2>Image Showcase</h2>
        <div class="mb-3">
            <button id="btn1" type="button" class="btn btn-primary">Image 1</button>
            <button id="btn2" type="button" class="btn btn-success">Image 2</button>
            <button id="btn3" type="button" class="btn btn-danger">Image 3</button>
        </div>
        <div class="image-container">
            <img id="img1" class="layered-img" src="https://images.unsplash.com/photo-1500534623283-312aade485b7?auto=format&fit=crop&w=500&q=80"
alt="Image 1">
            <img id="img2" class="layered-img" src="https://images.unsplash.com/photo-1511497584788-876760111969?auto=format&fit=crop&w=500&q=80"
alt="Image 2">
            <img id="img3" class="layered-img" src="https://images.unsplash.com/photo-1462331940025-496dfbfc7564?auto=format&fit=crop&w=500&q=80"
alt="Image 3">
        </div>
    </section>
    <script>
        document.addEventListener("DOMContentLoaded", function () {
            const img1 = document.getElementById("img1");
            const img2 = document.getElementById("img2");
            const img3 = document.getElementById("img3");
            function bringToFront(selectedImage) {
                img1.style.zIndex = "1";
                img2.style.zIndex = "1";
                img3.style.zIndex = "1";
                selectedImage.style.zIndex = "10";
            }
            document.getElementById("btn1").addEventListener("click", function () { bringToFront(img1); });
            document.getElementById("btn2").addEventListener("click", function () { bringToFront(img2); });
            document.getElementById("btn3").addEventListener("click", function () { bringToFront(img3); });
        });
    </script>
</body>
</html>
```

Code 4 — Live Textarea Editor (HTML + JavaScript)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Live Textarea Editor</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
    <style>
        #editor { min-height: 180px; font-size: 12px; color: black; font-weight: normal; font-style: normal; }
    </style>
</head>
<body>
    <div class="container mt-4">
        <h2 class="text-center mb-3">Live Textarea Editor</h2>
        <div class="row g-2 align-items-end mb-3">
            <div class="col-md-4">
                <label for="fontSizeSelect" class="form-label">Font Size</label>
                <select id="fontSizeSelect" class="form-select">
                    <option value="12px" selected>12px</option>
                    <option value="16px">16px</option>
                    <option value="20px">20px</option>
                </select>
            </div>
            <div class="col-md-4">
                <label for="fontColorSelect" class="form-label">Font Color</label>
                <select id="fontColorSelect" class="form-select">
                    <option value="black" selected>Black</option>
                    <option value="blue">Blue</option>
                    <option value="red">Red</option>
                </select>
            </div>
            <div class="col-md-4">
                <button type="button" id="boldBtn" class="btn btn-outline-dark">Bold</button>
                <button type="button" id="italicBtn" class="btn btn-outline-dark">Italic</button>
                <button type="button" id="resetBtn" class="btn btn-outline-secondary">Reset</button>
            </div>
        </div>
        <textarea id="editor" class="form-control" placeholder="Type something here..."></textarea>
    </div>
    <script>
        document.addEventListener("DOMContentLoaded", function () {
            const editor = document.getElementById("editor");
            const fontSizeSelect = document.getElementById("fontSizeSelect");
            const fontColorSelect = document.getElementById("fontColorSelect");
            const boldBtn = document.getElementById("boldBtn");
            const italicBtn = document.getElementById("italicBtn");
            const resetBtn = document.getElementById("resetBtn");
            fontSizeSelect.addEventListener("change", function () { editor.style.fontSize = fontSizeSelect.value; });
            fontColorSelect.addEventListener("change", function () { editor.style.color = fontColorSelect.value; });
            boldBtn.addEventListener("click", function () {
                boldBtn.classList.toggle("active");
                editor.style.fontWeight = boldBtn.classList.contains("active") ? "bold" : "normal";
            });
            italicBtn.addEventListener("click", function () {
                italicBtn.classList.toggle("active");
                editor.style.fontStyle = italicBtn.classList.contains("active") ? "italic" : "normal";
            });
            resetBtn.addEventListener("click", function () {
                fontSizeSelect.value = "12px";
                fontColorSelect.value = "black";
                editor.style.fontSize = "12px";
                editor.style.color = "black";
                editor.style.fontWeight = "normal";
```

```
                editor.style.fontStyle = "normal";
                boldBtn.classList.remove("active");
                italicBtn.classList.remove("active");
            });
        });
    </script>
</body>
</html>
```

Code 5 — Kids' Sketch Pad (HTML + JavaScript)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Kids Sketch Pad</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
    <style>
        #sketchPad { display: block; width: 500px; height: 300px; max-width: 100%; border: 1px solid black; background: white; cursor: crosshair; }
    </style>
</head>
<body>
    <div class="container mt-4">
        <h2>Kids' Sketch Pad</h2>
        <canvas id="sketchPad" width="500" height="300"></canvas>
        <div class="mt-3">
            <button id="redBtn" class="btn btn-danger" type="button">Red</button>
            <button id="blueBtn" class="btn btn-primary" type="button">Blue</button>
            <button id="greenBtn" class="btn btn-success" type="button">Green</button>
            <button id="eraserBtn" class="btn btn-secondary" type="button">Eraser</button>
            <button id="clearBtn" class="btn btn-warning" type="button">Clear Canvas</button>
        </div>
        <p class="mt-2">Current Color: <strong id="currentColor">Black</strong></p>
    </div>
    <script>
        document.addEventListener("DOMContentLoaded", function () {
            const canvas = document.getElementById("sketchPad");
            const ctx = canvas.getContext("2d");
            const redBtn = document.getElementById("redBtn");
            const blueBtn = document.getElementById("blueBtn");
            const greenBtn = document.getElementById("greenBtn");
            const eraserBtn = document.getElementById("eraserBtn");
            const clearBtn = document.getElementById("clearBtn");
            const currentColorDisplay = document.getElementById("currentColor");
            let drawing = false;
            ctx.strokeStyle = "#000000";
            ctx.lineWidth = 3;
            ctx.lineCap = "round";
            ctx.lineJoin = "round";
            function getPosition(event) {
                const rect = canvas.getBoundingClientRect();
                return { x: (event.clientX - rect.left) * canvas.width / rect.width, y: (event.clientY - rect.top) * canvas.height / rect.height };
            }
            canvas.addEventListener("mousedown", function (event) {
                const pos = getPosition(event);
                drawing = true;
                ctx.beginPath();
                ctx.moveTo(pos.x, pos.y);
            });
            canvas.addEventListener("mousemove", function (event) {
                if (!drawing) return;
                const pos = getPosition(event);
                ctx.lineTo(pos.x, pos.y);
                ctx.stroke();
            });
            function stopDrawing() { drawing = false; ctx.closePath(); }
            canvas.addEventListener("mouseup", stopDrawing);
            canvas.addEventListener("mouseout", stopDrawing);
            redBtn.addEventListener("click", function () { ctx.strokeStyle = "#ff0000"; currentColorDisplay.textContent = "Red"; });
            blueBtn.addEventListener("click", function () { ctx.strokeStyle = "#0000ff"; currentColorDisplay.textContent = "Blue"; });
            greenBtn.addEventListener("click", function () { ctx.strokeStyle = "#008000"; currentColorDisplay.textContent = "Green"; });
            eraserBtn.addEventListener("click", function () { ctx.strokeStyle = "#ffffff"; currentColorDisplay.textContent = "White (Eraser)"; });
            clearBtn.addEventListener("click", function () {
```

```
                ctx.clearRect(0, 0, canvas.width, canvas.height);
                ctx.strokeStyle = "#000000";
                currentColorDisplay.textContent = "Black";
            });
        });
    </script>
</body>
</html>
```

WEEK 7 — PROGRAMS WITH IDENTIFICATION TITLES

Code 1 — Student Attendance Tracker

```
1.cod 1
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Attendance Tracker</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <h1>Student Attendance Tracker</h1>

        <fieldset>
            <legend>Select Class Section</legend>
            <div class="radio-group">
                <label><input type="radio" name="section" value="A"> Section A</label>
                <label><input type="radio" name="section" value="B"> Section B</label>
                <label><input type="radio" name="section" value="C"> Section C</label>
            </div>
            <p>Selected Section: <span id="sectionDisplay">None</span></p>
        </fieldset>

        <fieldset>
            <legend>Mark Attendance</legend>
            <div class="student-grid">
                <label><input type="checkbox" id="student1" class="student-checkbox"> Arav Kumar</label>
                <label><input type="checkbox" id="student2" class="student-checkbox"> Bhavya Sharma</label>
                <label><input type="checkbox" id="student3" class="student-checkbox"> Chinmay Patel</label>
                <label><input type="checkbox" id="student4" class="student-checkbox"> Divya Nair</label>
                <label><input type="checkbox" id="student5" class="student-checkbox"> Eshan Mehta</label>
            </div>
        </fieldset>

        <select id="sectionSelect" class="d-none">
            <option value="">Select Section</option>
            <option value="A">A</option>
            <option value="B">B</option>
            <option value="C">C</option>
        </select>

        <button id="submitBtn">Calculate Attendance</button>

        <fieldset>
            <legend>Attendance Summary</legend>
            <div class="summary">
                <div class="card present">
                    <span>Present</span>
                    <strong id="presentCount">0</strong>
                </div>
                <div class="card absent">
                    <span>Absent</span>
                    <strong id="absentCount">0</strong>
                </div>
                <div class="card percentage">
                    <span>Attendance %</span>
                    <strong id="attendancePercent">0%</strong>
                </div>
            </div>
```

```
        </fieldset>
    </div>

    <script src="script.js"></script>
</body>
</html>
--------------------------------------------------------------------------------
* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: Arial, sans-serif;
    background: #f1f3f5;
    color: #222;
}

.container {
    width: 100%;
    max-width: 700px;
    margin: 30px auto;
    padding: 20px;
}

h1 {
    text-align: center;
    font-size: 24px;
    margin-bottom: 25px;
}

fieldset {
    background: white;
    border: 1px solid #d6d6d6;
    border-radius: 6px;
    padding: 20px;
    margin-bottom: 18px;
}

legend {
    font-weight: bold;
    padding: 0 8px;
}

.radio-group {
    display: flex;
    justify-content: space-between;
    gap: 15px;
    margin: 8px 0 12px;
}

.radio-group label,
.student-grid label {
    font-size: 14px;
}

#sectionDisplay {
    font-weight: bold;
}

.student-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 12px 20px;
}
```

```
button {
    display: block;
    margin: 20px auto;
    padding: 10px 24px;
    border: none;
    border-radius: 5px;
    background: #2868df;
    color: white;
    font-size: 14px;
    cursor: pointer;
}

button:hover {
    background: #1f55bd;
}

.summary {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 10px;
}

.card {
    text-align: center;
    color: white;
    padding: 14px 8px;
    border-radius: 5px;
}

.card span {
    display: block;
    font-size: 12px;
    margin-bottom: 6px;
}

.card strong {
    display: block;
    font-size: 22px;
}

.present {
    background: #16a34a;
}

.absent {
    background: #e52424;
}

.percentage {
    background: #2868df;
}

.d-none {
    display: none;
}

@media (max-width: 600px) {
    .student-grid {
        grid-template-columns: repeat(2, 1fr);
    }

    .radio-group {
        flex-direction: column;
    }

    .summary {
```

```
        grid-template-columns: 1fr;
    }
}
-----------------------------------------------------
document.addEventListener("DOMContentLoaded", function () {
    const sectionRadios = document.querySelectorAll('input[name="section"]');
    const studentCheckboxes = document.querySelectorAll(".student-checkbox");
    const sectionDisplay = document.getElementById("sectionDisplay");
    const submitBtn = document.getElementById("submitBtn");
    const presentCount = document.getElementById("presentCount");
    const absentCount = document.getElementById("absentCount");
    const attendancePercent = document.getElementById("attendancePercent");

    sectionRadios.forEach(function (radio) {
        radio.addEventListener("change", function () {
            sectionDisplay.textContent = radio.value;
        });
    });

    submitBtn.addEventListener("click", function () {
        let present = 0;

        for (let i = 0; i < studentCheckboxes.length; i++) {
            if (studentCheckboxes[i].checked) {
                present++;
            }
        }

        let selectedSection = "";
        let i = 0;

        while (i < sectionRadios.length) {
            if (sectionRadios[i].checked) {
                selectedSection = sectionRadios[i].value;
                break;
            }
            i++;
        }

        let absent = 0;
        let index = 0;

        if (studentCheckboxes.length > 0) {
            do {
                if (!studentCheckboxes[index].checked) {
                    absent++;
                }
                index++;
            } while (index < studentCheckboxes.length);
        }

        let percentage = 0;

        if (studentCheckboxes.length > 0) {
            percentage = (present / studentCheckboxes.length) * 100;
        }

        sectionDisplay.textContent = selectedSection || "None";
        presentCount.textContent = present;
        absentCount.textContent = absent;
        attendancePercent.textContent = percentage.toFixed(0) + "%";
    });
});
-------------------------------------------------------------------------
```

WEEK 7

Code 2 — Movie Genre Poll

```
2. cod //create script.js separately

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Movie Genre Poll</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <h1>Movie Genre Poll</h1>

        <fieldset>
            <legend>Select Your Age Group</legend>

            <div class="age-options">
                <label>
                    <input type="radio" name="ageGroup" value="teen">
                    Teen (13-17)
                </label>

                <label>
                    <input type="radio" name="ageGroup" value="adult">
                    Adult (18-40)
                </label>

                <label>
                    <input type="radio" name="ageGroup" value="senior">
                    Senior (40+)
                </label>
            </div>

            <p>Selected Age Group: <span id="ageDisplay">None</span></p>
        </fieldset>

        <fieldset>
            <legend>Choose Your Favourite Genres</legend>

            <div class="genre-options">
                <label>
                    <input type="checkbox" id="genre1" class="genre-checkbox" value="Action">
                    Action
                </label>

                <label>
                    <input type="checkbox" id="genre2" class="genre-checkbox" value="Comedy">
                    Comedy
                </label>

                <label>
                    <input type="checkbox" id="genre3" class="genre-checkbox" value="Drama">
                    Drama
                </label>

                <label>
                    <input type="checkbox" id="genre4" class="genre-checkbox" value="Thriller">
                    Thriller
```

```
                </label>

                <label>
                    <input type="checkbox" id="genre5" class="genre-checkbox" value="Romance">
                    Romance
                </label>

                <label>
                    <input type="checkbox" id="genre6" class="genre-checkbox" value="Sci-Fi">
                    Sci-Fi
                </label>
            </div>
        </fieldset>

        <div class="buttons">
            <button type="button" id="pollSubmitBtn">Submit Poll</button>
            <button type="button" id="pollResetBtn">Reset</button>
        </div>

        <fieldset>
            <legend>Poll Results</legend>

            <div class="results">
                <div class="result-card selected">
                    <span>Genres Selected</span>
                    <strong id="genreCount">0</strong>
                </div>

                <div class="result-card total">
                    <span>Total Genres Available</span>
                    <strong id="totalGenres">6</strong>
                </div>
            </div>

            <div class="selected-genres">
                <strong>Selected Genres:</strong>
                <p id="genreList">None</p>
            </div>
        </fieldset>
    </div>

    <script src="script.js"></script>
</body>
</html>
--------------------------------------------------------------------------------


* {
    box-sizing: border-box;
}

body {
    margin: 0;
    padding: 20px;
    font-family: Arial, sans-serif;
    background: #f1f3f5;
    color: #222;
}

.container {
    width: 100%;
    max-width: 650px;
    margin: 0 auto;
}

h1 {
```

```
    text-align: center;
    font-size: 22px;
    margin: 10px 0 20px;
}

fieldset {
    background: #ffffff;
    border: 1px solid #d8d8d8;
    border-radius: 6px;
    padding: 18px;
    margin-bottom: 16px;
}

legend {
    font-size: 13px;
    font-weight: bold;
    padding: 0 6px;
}

.age-options {
    display: flex;
    justify-content: space-between;
    gap: 15px;
    margin: 8px 0 12px;
}

.age-options label,
.genre-options label {
    font-size: 12px;
}

fieldset p {
    font-size: 12px;
    margin: 8px 0 0;
}

#ageDisplay {
    font-weight: bold;
}

.genre-options {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 12px 25px;
}

.buttons {
    display: flex;
    justify-content: center;
    gap: 10px;
    margin: 10px 0 16px;
}

button {
    border: none;
    border-radius: 5px;
    padding: 10px 20px;
    font-size: 12px;
    cursor: pointer;
}

#pollSubmitBtn {
    background: #2868df;
    color: white;
}
```

```
#pollResetBtn {
    background: #e1e4e8;
    color: #333;
}

.results {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 10px;
}

.result-card {
    color: white;
    text-align: center;
    padding: 14px;
    border-radius: 5px;
}

.result-card span {
    display: block;
    font-size: 10px;
    margin-bottom: 6px;
}

.result-card strong {
    display: block;
    font-size: 22px;
}

.selected {
    background: #2868df;
}

.total {
    background: #6b7280;
}

.selected-genres {
    margin-top: 14px;
    font-size: 12px;
}

.selected-genres p {
    margin: 7px 0 0;
}

@media (max-width: 600px) {
    .age-options {
        flex-direction: column;
    }

    .genre-options {
        grid-template-columns: repeat(2, 1fr);
    }

    .results {
        grid-template-columns: 1fr;
    }
}
-------------------------------------------------------------
document.addEventListener("DOMContentLoaded", function () {
    const ageRadios = document.querySelectorAll('input[type="radio"][name="ageGroup"]');
    const genreCheckboxes = document.querySelectorAll('input[type="checkbox"].genre-checkbox');

    const ageDisplay = document.getElementById("ageDisplay");
    const genreCount = document.getElementById("genreCount");
```

```
    const totalGenres = document.getElementById("totalGenres");
    const genreList = document.getElementById("genreList");

    const pollSubmitBtn = document.getElementById("pollSubmitBtn");
    const pollResetBtn = document.getElementById("pollResetBtn");

    totalGenres.textContent = genreCheckboxes.length;

    for (let i = 0; i < ageRadios.length; i++) {
        ageRadios[i].addEventListener("change", function () {
            ageDisplay.textContent = this.value;
        });
    }

    pollSubmitBtn.addEventListener("click", function () {
        let selectedCount = 0;

        for (let i = 0; i < genreCheckboxes.length; i++) {
            if (genreCheckboxes[i].checked) {
                selectedCount++;
            }
        }

        let selectedAge = "";
        let radioIndex = 0;

        while (radioIndex < ageRadios.length) {
            if (ageRadios[radioIndex].checked) {
                selectedAge = ageRadios[radioIndex].value;
                break;
            }
            radioIndex++;
        }

        let selectedGenres = [];
        let genreIndex = 0;

        if (genreCheckboxes.length > 0) {
            do {
                if (genreCheckboxes[genreIndex].checked) {
                    selectedGenres.push(genreCheckboxes[genreIndex].value);
                }

                genreIndex++;
            } while (genreIndex < genreCheckboxes.length);
        }

        ageDisplay.textContent = selectedAge || "None";
        genreCount.textContent = selectedCount;

        if (selectedGenres.length > 0) {
            genreList.textContent = selectedGenres.join(", ");
        } else {
            genreList.textContent = "None";
        }
    });

    pollResetBtn.addEventListener("click", function () {
        for (let i = 0; i < genreCheckboxes.length; i++) {
            genreCheckboxes[i].checked = false;
        }

        genreCount.textContent = "0";
        genreList.textContent = "None";
    });
});
```

WEEK 7

Code 3 — Student Quiz Evaluation System

```
3. cy //create script.js
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Quiz Evaluation System</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">

    <h1>Student Quiz Evaluation System</h1>

    <div class="left-panel">

        <label for="studentName">Student Name</label>
        <input type="text" id="studentName" placeholder="Enter your name">

        <fieldset>
            <legend>Select Subject</legend>

            <div class="subject-options">
                <label>
                    <input type="radio" name="subject" value="maths">
                    Maths
                </label>

                <label>
                    <input type="radio" name="subject" value="science">
                    Science
                </label>

                <label>
                    <input type="radio" name="subject" value="english">
                    English
                </label>
            </div>

            <p>Selected Subject: <span id="subjectDisplay">None</span></p>
        </fieldset>

        <fieldset>
            <legend>Topics Covered</legend>

            <div class="topic-options">
                <label>
                    <input type="checkbox" class="topic-checkbox" value="Algebra">
                    Algebra
                </label>

                <label>
                    <input type="checkbox" class="topic-checkbox" value="Geometry">
                    Geometry
                </label>

                <label>
                    <input type="checkbox" class="topic-checkbox" value="Biology">
```

```
                Biology
            </label>

            <label>
                <input type="checkbox" class="topic-checkbox" value="Grammar">
                Grammar
            </label>
        </div>
    </fieldset>

    <div class="buttons">
        <button type="button" id="submitQuizBtn">Submit Quiz</button>
        <button type="button" id="resetQuizBtn">Reset</button>
    </div>

    <fieldset class="result-box">
        <legend>Result</legend>

        <div class="result-row">
            <span>Score</span>
            <strong id="quizScore">0</strong>
        </div>

        <div class="result-row">
            <span>Grade</span>
            <strong id="quizGrade">-</strong>
        </div>

        <div class="result-row">
            <span>Remark</span>
            <strong id="quizRemark">-</strong>
        </div>

        <div class="result-row">
            <span>Topics</span>
            <strong id="topicList">-</strong>
        </div>
    </fieldset>

</div>

<div class="right-panel">

    <fieldset>
        <legend>Question 1</legend>

        <p>What is 2 + 2?</p>

        <label>
            <input type="radio" name="q1" value="a">
            4
        </label>

        <label>
            <input type="radio" name="q1" value="b">
            3
        </label>

        <label>
            <input type="radio" name="q1" value="c">
            5
        </label>
    </fieldset>

    <fieldset>
        <legend>Question 2</legend>
```

```
            <p>Which planet is closest to the Sun?</p>

            <label>
                <input type="radio" name="q2" value="a">
                Earth
            </label>

            <label>
                <input type="radio" name="q2" value="b">
                Mercury
            </label>

            <label>
                <input type="radio" name="q2" value="c">
                Mars
            </label>
        </fieldset>

        <fieldset>
            <legend>Question 3</legend>

            <p>How many vowels are there in the English alphabet?</p>

            <label>
                <input type="radio" name="q3" value="a">
                4
            </label>

            <label>
                <input type="radio" name="q3" value="b">
                6
            </label>

            <label>
                <input type="radio" name="q3" value="c">
                5
            </label>
        </fieldset>

    </div>

</div>

<script src="script.js"></script>

</body>
</html>
-------------------------------------------------------
* {
    box-sizing: border-box;
}

body {
    margin: 0;
    padding: 20px;
    font-family: Arial, sans-serif;
    background: #f1f3f5;
    color: #222;
}

.container {
    width: 100%;
    max-width: 900px;
    margin: 0 auto;
}
```

```
h1 {
    text-align: center;
    font-size: 22px;
    margin: 10px 0 20px;
}

.left-panel,
.right-panel {
    width: 100%;
}

.container {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 18px;
}

.container h1 {
    grid-column: 1 / -1;
}

.left-panel {
    grid-column: 1;
}

.right-panel {
    grid-column: 2;
}

fieldset {
    background: #ffffff;
    border: 1px solid #d8d8d8;
    border-radius: 6px;
    padding: 14px;
    margin: 0 0 12px;
}

legend {
    font-size: 12px;
    font-weight: bold;
    padding: 0 5px;
}

#studentName {
    width: 100%;
    padding: 8px;
    border: 1px solid #ccc;
    border-radius: 4px;
    margin: 5px 0 12px;
    font-size: 12px;
}

.subject-options {
    display: flex;
    gap: 12px;
    margin: 8px 0;
}

.subject-options label,
.topic-options label,
.right-panel label {
    font-size: 11px;
}

fieldset p {
    font-size: 11px;
```

```
    margin: 7px 0 0;
}

#subjectDisplay {
    font-weight: bold;
}

.topic-options {
    display: flex;
    gap: 12px;
    flex-wrap: wrap;
}

.right-panel fieldset p {
    margin-bottom: 8px;
}

.right-panel label {
    margin-right: 8px;
}

.buttons {
    display: flex;
    gap: 8px;
    margin-bottom: 12px;
}

button {
    flex: 1;
    border: none;
    border-radius: 4px;
    padding: 9px;
    font-size: 11px;
    cursor: pointer;
}

#submitQuizBtn {
    background: #2868df;
    color: white;
}

#resetQuizBtn {
    background: #e1e4e8;
    color: #333;
}

.result-row {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 7px 0;
    border-bottom: 1px solid #eeeeee;
    font-size: 11px;
}

.result-row:last-child {
    border-bottom: none;
}

.result-row strong {
    color: #2868df;
}

@media (max-width: 700px) {
    .container {
        display: block;
```

```
    }

    .left-panel,
    .right-panel {
        width: 100%;
    }
}
-----------------------------------------------------------------------------------------------------------
document.addEventListener("DOMContentLoaded", function () {
    const studentName = document.getElementById("studentName");
    const subjectRadios = document.querySelectorAll('input[type="radio"][name="subject"]');

    const q1Radios = document.querySelectorAll('input[type="radio"][name="q1"]');
    const q2Radios = document.querySelectorAll('input[type="radio"][name="q2"]');
    const q3Radios = document.querySelectorAll('input[type="radio"][name="q3"]');

    const topicCheckboxes = document.querySelectorAll('input[type="checkbox"].topic-checkbox');

    const subjectDisplay = document.getElementById("subjectDisplay");
    const topicList = document.getElementById("topicList");

    const quizScore = document.getElementById("quizScore");
    const quizGrade = document.getElementById("quizGrade");
    const quizRemark = document.getElementById("quizRemark");

    const submitQuizBtn = document.getElementById("submitQuizBtn");
    const resetQuizBtn = document.getElementById("resetQuizBtn");

    for (let i = 0; i < subjectRadios.length; i++) {
        subjectRadios[i].addEventListener("change", function () {
            subjectDisplay.textContent = this.value;
        });
    }

    submitQuizBtn.addEventListener("click", function () {
        const questions = [q1Radios, q2Radios, q3Radios];
        const correctAnswers = ["a", "b", "c"];

        let score = 0;

        for (let i = 0; i < questions.length; i++) {
            for (let j = 0; j < questions[i].length; j++) {
                if (questions[i][j].checked && questions[i][j].value === correctAnswers[i]) {
                    score++;
                }
            }
        }

        let selectedSubject = "";
        let subjectIndex = 0;

        while (subjectIndex < subjectRadios.length) {
            if (subjectRadios[subjectIndex].checked) {
                selectedSubject = subjectRadios[subjectIndex].value;
                break;
            }
            subjectIndex++;
        }

        let grade = "-";

        if (score === 3) {
            grade = "A";
        } else if (score === 2) {
            grade = "B";
        } else if (score === 1) {
```

```
        grade = "C";
    } else {
        grade = "F";
    }

    let remark = "-";
    let remarkIndex = 0;

    do {
        if (score === 3 && remarkIndex === 0) {
            remark = "Excellent";
            break;
        }

        if (score === 2 && remarkIndex === 1) {
            remark = "Good";
            break;
        }

        if (score === 1 && remarkIndex === 2) {
            remark = "Needs Improvement";
            break;
        }

        if (score === 0 && remarkIndex === 3) {
            remark = "Please Try Again";
            break;
        }

        remarkIndex++;
    } while (remarkIndex < 4);

    let selectedTopics = [];
    let topicIndex = 0;

    if (topicCheckboxes.length > 0) {
        do {
            if (topicCheckboxes[topicIndex].checked) {
                selectedTopics.push(topicCheckboxes[topicIndex].value);
            }

            topicIndex++;
        } while (topicIndex < topicCheckboxes.length);
    }

    subjectDisplay.textContent = selectedSubject || "None";
    quizScore.textContent = score;
    quizGrade.textContent = grade;
    quizRemark.textContent = remark;
    topicList.textContent = selectedTopics.length > 0
        ? selectedTopics.join(", ")
        : "-";
});

resetQuizBtn.addEventListener("click", function () {
    studentName.value = "";

    for (let i = 0; i < subjectRadios.length; i++) {
        subjectRadios[i].checked = false;
    }

    for (let i = 0; i < q1Radios.length; i++) {
        q1Radios[i].checked = false;
    }

    for (let i = 0; i < q2Radios.length; i++) {
```

```
            q2Radios[i].checked = false;
        }

        for (let i = 0; i < q3Radios.length; i++) {
            q3Radios[i].checked = false;
        }

        for (let i = 0; i < topicCheckboxes.length; i++) {
            topicCheckboxes[i].checked = false;
        }

        subjectDisplay.textContent = "None";
        quizScore.textContent = "0";
        quizGrade.textContent = "-";
        quizRemark.textContent = "-";
        topicList.textContent = "-";
    });
});
--------------------------------------------------------------------------------
```

WEEK 7

Code 4 — Shopping Bill Calculator

```
4.pah 1
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Shopping Bill Calculator</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">

    <h1>Shopping Bill Calculator</h1>

    <fieldset>
        <legend>Select Items</legend>

        <div class="item-grid">
            <label>
                <input type="checkbox" id="item1" class="item-checkbox" data-price="120" value="Rice">
                Rice — Rs.120
            </label>

            <label>
                <input type="checkbox" id="item2" class="item-checkbox" data-price="95" value="Oil">
                Oil — Rs.95
            </label>

            <label>
                <input type="checkbox" id="item3" class="item-checkbox" data-price="50" value="Sugar">
                Sugar — Rs.50
            </label>

            <label>
                <input type="checkbox" id="item4" class="item-checkbox" data-price="60" value="Milk">
                Milk — Rs.60
            </label>

            <label>
                <input type="checkbox" id="item5" class="item-checkbox" data-price="40" value="Bread">
                Bread — Rs.40
            </label>

            <label>
                <input type="checkbox" id="item6" class="item-checkbox" data-price="75" value="Butter">
                Butter — Rs.75
            </label>
        </div>
    </fieldset>

    <fieldset>
        <legend>Select Payment Method</legend>

        <div class="payment-options">
            <label>
                <input type="radio" name="paymentMethod" value="cash">
                Cash (No Discount)
            </label>
```

```
            <label>
                <input type="radio" name="paymentMethod" value="card">
                Card (5% Discount)
            </label>

            <label>
                <input type="radio" name="paymentMethod" value="upi">
                UPI (10% Discount)
            </label>
        </div>

        <p>Selected Payment: <span id="paymentDisplay">None</span></p>
    </fieldset>

    <div class="buttons">
        <button type="button" id="calculateBtn">Calculate Bill</button>
        <button type="button" id="resetBillBtn">Reset</button>
    </div>

    <fieldset>
        <legend>Bill Summary</legend>

        <div class="summary">
            <div class="summary-card">
                <span>Total Amount (Rs.)</span>
                <strong id="totalAmount">0</strong>
            </div>

            <div class="summary-card">
                <span>Discount (Rs.)</span>
                <strong id="discountAmount">0</strong>
            </div>

            <div class="summary-card">
                <span>Final Amount (Rs.)</span>
                <strong id="finalAmount">0</strong>
            </div>
        </div>

        <div class="selected-items">
            <strong>Selected Items:</strong>
            <p id="itemList">None</p>
        </div>
    </fieldset>

</div>

<script src="script.js"></script>

</body>
</html>
--------------------------------------------------------------------------------
* {
    box-sizing: border-box;
}

body {
    margin: 0;
    padding: 20px;
    font-family: Arial, sans-serif;
    background: #f1f3f5;
    color: #222;
}

.container {
    width: 100%;
```

```
    max-width: 700px;
    margin: 0 auto;
}

h1 {
    text-align: center;
    font-size: 22px;
    margin: 10px 0 20px;
}

fieldset {
    background: #ffffff;
    border: 1px solid #d8d8d8;
    border-radius: 6px;
    padding: 18px;
    margin-bottom: 14px;
}

legend {
    font-size: 12px;
    font-weight: bold;
    padding: 0 6px;
}

.item-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 12px 20px;
}

.item-grid label,
.payment-options label {
    font-size: 11px;
}

.payment-options {
    display: flex;
    justify-content: space-between;
    gap: 12px;
    margin: 8px 0 10px;
}

fieldset p {
    font-size: 11px;
    margin: 8px 0 0;
}

#paymentDisplay {
    font-weight: bold;
}

.buttons {
    display: flex;
    justify-content: center;
    gap: 10px;
    margin: 16px 0;
}

button {
    border: none;
    border-radius: 5px;
    padding: 10px 22px;
    font-size: 11px;
    cursor: pointer;
}
```

```
#calculateBtn {
    background: #2868df;
    color: white;
}

#resetBillBtn {
    background: #e1e4e8;
    color: #333;
}

.summary {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 10px;
}

.summary-card {
    background: #2868df;
    color: white;
    text-align: center;
    padding: 14px 8px;
    border-radius: 5px;
}

.summary-card span {
    display: block;
    font-size: 9px;
    margin-bottom: 6px;
}

.summary-card strong {
    display: block;
    font-size: 22px;
}

.selected-items {
    margin-top: 14px;
    font-size: 11px;
}

.selected-items p {
    margin: 7px 0 0;
}

@media (max-width: 600px) {
    .item-grid {
        grid-template-columns: repeat(2, 1fr);
    }

    .payment-options {
        flex-direction: column;
    }

    .summary {
        grid-template-columns: 1fr;
    }
}
-----------------------------------------------------------
document.addEventListener("DOMContentLoaded", function () {
    const itemCheckboxes = document.querySelectorAll('input[type="checkbox"].item-checkbox');
    const paymentRadios = document.querySelectorAll('input[type="radio"][name="paymentMethod"]');

    const paymentDisplay = document.getElementById("paymentDisplay");
    const totalAmount = document.getElementById("totalAmount");
    const discountAmount = document.getElementById("discountAmount");
    const finalAmount = document.getElementById("finalAmount");
```

```
const itemList = document.getElementById("itemList");

const calculateBtn = document.getElementById("calculateBtn");
const resetBillBtn = document.getElementById("resetBillBtn");

for (let i = 0; i < paymentRadios.length; i++) {
    paymentRadios[i].addEventListener("change", function () {
        paymentDisplay.textContent = this.value;
    });
}

calculateBtn.addEventListener("click", function () {
    let total = 0;

    for (let i = 0; i < itemCheckboxes.length; i++) {
        if (itemCheckboxes[i].checked) {
            total += parseFloat(itemCheckboxes[i].getAttribute("data-price")) || 0;
        }
    }

    let paymentMethod = "";
    let paymentIndex = 0;

    while (paymentIndex < paymentRadios.length) {
        if (paymentRadios[paymentIndex].checked) {
            paymentMethod = paymentRadios[paymentIndex].value;
            break;
        }
        paymentIndex++;
    }

    let discountRate = 0;

    if (paymentMethod === "upi") {
        discountRate = 0.10;
    } else if (paymentMethod === "card") {
        discountRate = 0.05;
    }

    const discount = parseFloat((total * discountRate).toFixed(2));
    const final = parseFloat((total - discount).toFixed(2));

    let selectedItems = [];
    let itemIndex = 0;

    if (itemCheckboxes.length > 0) {
        do {
            if (itemCheckboxes[itemIndex].checked) {
                selectedItems.push(itemCheckboxes[itemIndex].value);
            }

            itemIndex++;
        } while (itemIndex < itemCheckboxes.length);
    }

    totalAmount.textContent = total;
    discountAmount.textContent = discount;
    finalAmount.textContent = final;

    paymentDisplay.textContent = paymentMethod || "None";
    itemList.textContent = selectedItems.length > 0
        ? selectedItems.join(", ")
        : "None";
});

resetBillBtn.addEventListener("click", function () {
```

```
        for (let i = 0; i < itemCheckboxes.length; i++) {
            itemCheckboxes[i].checked = false;
        }

        for (let i = 0; i < paymentRadios.length; i++) {
            paymentRadios[i].checked = false;
        }

        totalAmount.textContent = "0";
        discountAmount.textContent = "0";
        finalAmount.textContent = "0";
        paymentDisplay.textContent = "None";
        itemList.textContent = "None";
    });
});
--------------------------------------------------------------------------------
```

WEEK 7

Code 5 — Fitness Activity Planner

```
5. pah2
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Fitness Activity Planner</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">

    <h1>Fitness Activity Planner</h1>

    <fieldset>
        <legend>Select Fitness Goal</legend>

        <div class="goal-options">
            <label>
                <input type="radio" name="fitnessGoal" value="weightloss">
                Weight Loss (400 cal/day)
            </label>

            <label>
                <input type="radio" name="fitnessGoal" value="muscle">
                Muscle Gain (300 cal/day)
            </label>

            <label>
                <input type="radio" name="fitnessGoal" value="stamina">
                Stamina (350 cal/day)
            </label>
        </div>

        <p>Selected Goal: <span id="goalDisplay">None</span></p>
    </fieldset>

    <fieldset>
        <legend>Select Workout Days</legend>

        <div class="day-grid">
            <label>
                <input type="checkbox" id="day1" class="day-checkbox" value="Monday">
                Monday
            </label>

            <label>
                <input type="checkbox" id="day2" class="day-checkbox" value="Tuesday">
                Tuesday
            </label>

            <label>
                <input type="checkbox" id="day3" class="day-checkbox" value="Wednesday">
                Wednesday
            </label>

            <label>
                <input type="checkbox" id="day4" class="day-checkbox" value="Thursday">
```

```
                Thursday
            </label>

            <label>
                <input type="checkbox" id="day5" class="day-checkbox" value="Friday">
                Friday
            </label>

            <label>
                <input type="checkbox" id="day6" class="day-checkbox" value="Saturday">
                Saturday
            </label>

            <label>
                <input type="checkbox" id="day7" class="day-checkbox" value="Sunday">
                Sunday
            </label>
        </div>
    </fieldset>

    <div class="buttons">
        <button type="button" id="calcPlanBtn">Calculate Plan</button>
        <button type="button" id="resetPlanBtn">Reset</button>
    </div>

    <fieldset>
        <legend>Weekly Summary</legend>

        <div class="summary">
            <div class="summary-card blue">
                <span>Workout Days</span>
                <strong id="workoutDays">0</strong>
            </div>

            <div class="summary-card green">
                <span>Calories Burned</span>
                <strong id="caloriesBurned">0</strong>
            </div>

            <div class="summary-card orange">
                <span>Rest Days</span>
                <strong id="restDays">0</strong>
            </div>
        </div>
    </fieldset>

</div>

<script src="script.js"></script>

</body>
</html>
-----------------------------------------------------------------

* {
    box-sizing: border-box;
}

body {
    margin: 0;
    padding: 20px;
    font-family: Arial, sans-serif;
    background: #f1f3f5;
    color: #222;
}
```

```
.container {
    width: 100%;
    max-width: 650px;
    margin: 0 auto;
}

h1 {
    text-align: center;
    font-size: 22px;
    margin: 10px 0 20px;
}

fieldset {
    background: #ffffff;
    border: 1px solid #d8d8d8;
    border-radius: 6px;
    padding: 18px;
    margin-bottom: 14px;
}

legend {
    font-size: 12px;
    font-weight: bold;
    padding: 0 6px;
}

.goal-options {
    display: flex;
    justify-content: space-between;
    gap: 12px;
    margin: 8px 0;
}

.goal-options label,
.day-grid label {
    font-size: 11px;
}

fieldset p {
    font-size: 11px;
    margin: 8px 0 0;
}

#goalDisplay {
    font-weight: bold;
}

.day-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 10px 20px;
}

.buttons {
    display: flex;
    justify-content: center;
    gap: 10px;
    margin: 16px 0;
}

button {
    border: none;
    border-radius: 5px;
    padding: 10px 22px;
    font-size: 11px;
    cursor: pointer;
```

```
}

#calcPlanBtn {
    background: #2868df;
    color: white;
}

#resetPlanBtn {
    background: #e1e4e8;
    color: #333;
}

.summary {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 10px;
}

.summary-card {
    color: white;
    text-align: center;
    padding: 14px 8px;
    border-radius: 5px;
}

.summary-card span {
    display: block;
    font-size: 9px;
    margin-bottom: 6px;
}

.summary-card strong {
    display: block;
    font-size: 22px;
}

.blue {
    background: #2868df;
}

.green {
    background: #16a34a;
}

.orange {
    background: #ea580c;
}

@media (max-width: 600px) {
    .goal-options {
        flex-direction: column;
    }

    .day-grid {
        grid-template-columns: repeat(2, 1fr);
    }

    .summary {
        grid-template-columns: 1fr;
    }
}
------------------------------------------------------------
document.addEventListener("DOMContentLoaded", function () {
    const goalRadios = document.querySelectorAll('input[type="radio"][name="fitnessGoal"]');
    const dayCheckboxes = document.querySelectorAll('input[type="checkbox"].day-checkbox');
```

```
const goalDisplay = document.getElementById("goalDisplay");
const workoutDays = document.getElementById("workoutDays");
const caloriesBurned = document.getElementById("caloriesBurned");
const restDays = document.getElementById("restDays");

const calcPlanBtn = document.getElementById("calcPlanBtn");
const resetPlanBtn = document.getElementById("resetPlanBtn");

for (let i = 0; i < goalRadios.length; i++) {
    goalRadios[i].addEventListener("change", function () {
        goalDisplay.textContent = this.value;
    });
}

calcPlanBtn.addEventListener("click", function () {
    let selectedDays = 0;

    for (let i = 0; i < dayCheckboxes.length; i++) {
        if (dayCheckboxes[i].checked) {
            selectedDays++;
        }
    }

    let selectedGoal = "";
    let goalIndex = 0;

    while (goalIndex < goalRadios.length) {
        if (goalRadios[goalIndex].checked) {
            selectedGoal = goalRadios[goalIndex].value;
            break;
        }
        goalIndex++;
    }

    let caloriesPerDay = 0;

    if (selectedGoal === "weightloss") {
        caloriesPerDay = 400;
    } else if (selectedGoal === "muscle") {
        caloriesPerDay = 300;
    } else if (selectedGoal === "stamina") {
        caloriesPerDay = 350;
    }

    let totalCalories = 0;
    let dayIndex = 0;

    if (selectedDays > 0) {
        do {
            if (dayCheckboxes[dayIndex].checked) {
                totalCalories += caloriesPerDay;
            }

            dayIndex++;
        } while (dayIndex < dayCheckboxes.length);
    }

    const remainingDays = 7 - selectedDays;

    workoutDays.textContent = selectedDays;
    caloriesBurned.textContent = totalCalories;
    restDays.textContent = remainingDays;

    goalDisplay.textContent = selectedGoal || "None";
});
```

```
    resetPlanBtn.addEventListener("click", function () {
        for (let i = 0; i < dayCheckboxes.length; i++) {
            dayCheckboxes[i].checked = false;
        }

        for (let i = 0; i < goalRadios.length; i++) {
            goalRadios[i].checked = false;
        }

        goalDisplay.textContent = "None";
        workoutDays.textContent = "0";
        caloriesBurned.textContent = "0";
        restDays.textContent = "0";
    });
});
----------------------------------------------------------------------
```

WEEK 8 — PROGRAMS WITH IDENTIFICATION TITLES

Code 1 — Employee Directory (React)

```
Week 8 cod1
1. src/App.js
import React, { Component } from 'react';
import './App.css';

class App extends Component {
  constructor(props) {
    super(props);
    this.state = {
      query: '',
      employees: [
        'John Doe',
        'Jane Smith',
        'Mike Johnson',
        'Emily Davis',
        'James Brown',
        'Jennifer Wilson',
        'Paul Garcia',
        'Laura Martinez'
      ],
      filteredEmployees: [
        'John Doe',
        'Jane Smith',
        'Mike Johnson',
        'Emily Davis',
        'James Brown',
        'Jennifer Wilson',
        'Paul Garcia',
        'Laura Martinez'
      ]
    };
  }

  componentDidUpdate(prevProps, prevState) {
    if (prevState.query !== this.state.query) {
      this.filterEmployees();
    }
  }

  handleSearch = (event) => {
    this.setState({ query: event.target.value });
  };

  filterEmployees = () => {
    const filteredEmployees = this.state.employees.filter((employee) =>
      employee.toLowerCase().includes(this.state.query.toLowerCase())
    );

    this.setState({ filteredEmployees });
  };

  render() {
    return (
      <div className="App">
        <h1>Employee Directory</h1>

        <input
          type="text"
          placeholder="Search employees..."
```

```
          value={this.state.query}
          onChange={this.handleSearch}
        />

        <div className="employee-list">
          {this.state.filteredEmployees.map((employee, index) => (
            <div className="employee" key={index}>
              {employee}
            </div>
          ))}
        </div>
      </div>
    );
  }
}

export default App;
```

---

2. src/App.css

```
.App {
  min-height: 100vh;
  background-color: #f5f5f5;
  text-align: center;
  padding-top: 25px;
  box-sizing: border-box;
  font-family: Arial, sans-serif;
}

h1 {
  color: #333;
  margin-bottom: 32px;
}

input {
  width: 326px;
  padding: 8px 10px;
  border: 1px solid #ccc;
  border-radius: 3px;
  font-size: 14px;
  box-sizing: border-box;
}

.employee-list {
  width: 310px;
  margin: 16px auto;
}

.employee {
  background-color: white;
  border: 1px solid #ddd;
  padding: 8px;
  margin-bottom: 8px;
  font-size: 14px;
  color: #333;
  border-radius: 2px;
}
```

WEEK 8

Code 2 — Stopwatch (React)

```
2. cod
src/App.js
import React from 'react';
import './App.css';
import Stopwatch from './Stopwatch';

function App() {
  return (
    <div className="App">
      <Stopwatch />
    </div>
  );
}

export default App;
----------------------------------------------------------------
src/Stopwatch.jsx
import React, { Component } from 'react';

class Stopwatch extends Component {
  constructor(props) {
    super(props);
    this.state = {
      time: 0,
      isRunning: false,
      laps: []
    };
    this.interval = null;
    this.startTime = 0;
  }

  componentDidMount() {
    this.interval = null;
  }

  componentDidUpdate(prevProps, prevState) {
    if (!prevState.isRunning && this.state.isRunning) {
      this.startTime = Date.now() - this.state.time;
      this.interval = setInterval(() => {
        this.setState({
          time: Date.now() - this.startTime
        });
      }, 10);
    }

    if (prevState.isRunning && !this.state.isRunning && this.interval) {
      clearInterval(this.interval);
      this.interval = null;
    }
  }

  componentWillUnmount() {
    if (this.interval) {
      clearInterval(this.interval);
    }
  }

  formatTime = (milliseconds) => {
    const minutes = Math.floor(milliseconds / 60000);
```

```
  const seconds = Math.floor((milliseconds % 60000) / 1000);
  const centiseconds = Math.floor((milliseconds % 1000) / 10);

  return `${String(minutes).padStart(2, '0')}:${String(seconds).padStart(2, '0')}.${String(centiseconds).padStart(2, '0')}`;
};

handleStart = () => {
  this.setState({
    isRunning: true
  });
};

handleStop = () => {
  this.setState((prevState) => ({
    isRunning: false,
    laps: [...prevState.laps, prevState.time]
  }));
};

handleReset = () => {
  this.setState({
    time: 0,
    isRunning: false,
    laps: []
  });
};

handleLap = () => {
  this.setState((prevState) => ({
    laps: [...prevState.laps, prevState.time]
  }));
};

render() {
  const { time, isRunning, laps } = this.state;

  return (
    <div className="stopwatch">
      <h2>Stopwatch</h2>

      <div className="time">
        {this.formatTime(time)}
      </div>

      <div className="buttons">
        <button
          className="start"
          onClick={this.handleStart}
          disabled={isRunning}
        >
          Start
        </button>

        <button
          className="stop"
          onClick={this.handleStop}
          disabled={!isRunning}
        >
          Stop
        </button>

        <button
          className="reset"
          onClick={this.handleReset}
          disabled={isRunning}
        >
```

```
            Reset
          </button>
        </div>

        <button
          className="lap"
          onClick={this.handleLap}
          disabled={!isRunning}
        >
          Add Lap
        </button>

        <div className="laps">
          {laps.map((lap, index) => (
            <div className="lap-item" key={index}>
              {this.formatTime(lap)}
            </div>
          ))}
        </div>
      </div>
    );
  }
}

export default Stopwatch;
--------------------------------------------------------------------------------
src/App.css
.App {
  text-align: center;
  min-height: 100vh;
  background-color: #f5f5f5;
  padding-top: 110px;
  box-sizing: border-box;
}

.stopwatch {
  width: 205px;
  min-height: 155px;
  margin: 0 auto;
  padding: 20px;
  border: 1px solid #555;
  border-radius: 4px;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.2);
  background-color: #f7f7f7;
  box-sizing: border-box;
}

.stopwatch h2 {
  font-size: 13px;
  margin: 0 0 8px;
  color: #333;
}

.time {
  font-size: 20px;
  color: #333;
  margin-bottom: 12px;
}

.buttons {
  display: flex;
  justify-content: center;
  gap: 6px;
}

button {
```

```
  border: none;
  color: white;
  padding: 5px 10px;
  font-size: 9px;
  cursor: pointer;
  border-radius: 2px;
}

button:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

.start {
  background-color: #28a745;
}

.stop {
  background-color: #dc3545;
}

.reset {
  background-color: #17a2b8;
}

.lap {
  background-color: #ffc107;
  color: #333;
  margin-top: 6px;
}

.laps {
  margin-top: 10px;
}

.lap-item {
  font-size: 11px;
  margin: 3px 0;
  color: #333;
}
--------------------------------------------------------------------------
```

WEEK 8

Code 3 — Kids' Sketch Pad

```
week 8 cy
html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Suresh's Kids' Sketch Pad</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <canvas id="sketchPad" width="500" height="300"></canvas>

  <div class="content">
    <h2>Suresh's Kids' Sketch Pad</h2>

    <div class="controls">
      <button id="defaultBtn">Default</button>
      <button id="redBtn">Red</button>
      <button id="blueBtn">Blue</button>
      <button id="greenBtn">Green</button>
      <button id="eraserBtn">Eraser</button>
      <button id="clearBtn">Clear Canvas</button>
    </div>

    <div id="currentColorDisplay">
      Current Drawing Color: <span id="currentColor">Black</span>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>
--------------------------------------------------------------------------------
css
html,
body {
  margin: 0;
  padding: 0;
  width: 100%;
  min-height: 100%;
  font-family: Arial, sans-serif;
  background: white;
}

#sketchPad {
  position: absolute;
  top: 0;
  left: 0;
  width: 500px;
  height: 300px;
  border: 1px solid #999;
  background: white;
  box-sizing: border-box;
  cursor: crosshair;
}

.content {
```

```
  position: absolute;
  top: 315px;
  left: 0;
  width: 500px;
  text-align: center;
}

h2 {
  margin: 0 0 12px;
  font-size: 20px;
  font-weight: 500;
  color: #222;
}

.controls {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 6px;
}

.controls button {
  border: none;
  border-radius: 4px;
  padding: 6px 10px;
  font-size: 11px;
  color: white;
  cursor: pointer;
}

#defaultBtn {
  background-color: #212529;
}

#redBtn {
  background-color: #dc3545;
}

#blueBtn {
  background-color: #0d6efd;
}

#greenBtn {
  background-color: #198754;
}

#eraserBtn {
  background-color: #6c757d;
}

#clearBtn {
  background-color: #ffc107;
  color: #222;
}

#currentColorDisplay {
  margin-top: 10px;
  font-size: 13px;
  color: #555;
}

#currentColor {
  font-weight: bold;
  color: #222;
}
-----------------------------------------------------------------
```

```
js
const canvas = document.getElementById('sketchPad');
const ctx = canvas.getContext('2d');

const defaultBtn = document.getElementById('defaultBtn');
const redBtn = document.getElementById('redBtn');
const blueBtn = document.getElementById('blueBtn');
const greenBtn = document.getElementById('greenBtn');
const eraserBtn = document.getElementById('eraserBtn');
const clearBtn = document.getElementById('clearBtn');
const currentColor = document.getElementById('currentColor');

let drawing = false;

ctx.strokeStyle = '#000000';
ctx.lineWidth = 3;
ctx.lineCap = 'round';
ctx.lineJoin = 'round';

ctx.fillStyle = '#ffffff';
ctx.fillRect(0, 0, canvas.width, canvas.height);

function startDrawing(event) {
  const rect = canvas.getBoundingClientRect();
  const x = event.clientX - rect.left;
  const y = event.clientY - rect.top;

  if (x >= 0 && x <= canvas.width && y >= 0 && y <= canvas.height) {
    drawing = true;
    ctx.beginPath();
    ctx.moveTo(x, y);
  }
}

function draw(event) {
  if (!drawing) {
    return;
  }

  const rect = canvas.getBoundingClientRect();
  const x = event.clientX - rect.left;
  const y = event.clientY - rect.top;

  if (x >= 0 && x <= canvas.width && y >= 0 && y <= canvas.height) {
    ctx.lineTo(x, y);
    ctx.stroke();
  }
}

function stopDrawing() {
  drawing = false;
  ctx.closePath();
}

defaultBtn.addEventListener('click', function() {
  ctx.strokeStyle = '#000000';
  currentColor.textContent = 'Black';
});

redBtn.addEventListener('click', function() {
  ctx.strokeStyle = '#ff0000';
  currentColor.textContent = 'Red';
});

blueBtn.addEventListener('click', function() {
  ctx.strokeStyle = '#0000ff';
```

```
  currentColor.textContent = 'Blue';
});

greenBtn.addEventListener('click', function() {
  ctx.strokeStyle = '#008000';
  currentColor.textContent = 'Green';
});

eraserBtn.addEventListener('click', function() {
  ctx.strokeStyle = '#ffffff';
  currentColor.textContent = 'White (Eraser)';
});

clearBtn.addEventListener('click', function() {
  ctx.clearRect(0, 0, canvas.width, canvas.height);

  ctx.fillStyle = '#ffffff';
  ctx.fillRect(0, 0, canvas.width, canvas.height);

  if (currentColor.textContent === 'Red') {
    ctx.strokeStyle = '#ff0000';
  } else if (currentColor.textContent === 'Blue') {
    ctx.strokeStyle = '#0000ff';
  } else if (currentColor.textContent === 'Green') {
    ctx.strokeStyle = '#008000';
  } else if (currentColor.textContent === 'White (Eraser)') {
    ctx.strokeStyle = '#ffffff';
  } else {
    ctx.strokeStyle = '#000000';
  }
});

canvas.addEventListener('mousedown', startDrawing);
canvas.addEventListener('mousemove', draw);
canvas.addEventListener('mouseup', stopDrawing);
canvas.addEventListener('mouseout', stopDrawing);
--------------------------------------------------------------------------------
```

WEEK 8

Code 4 — Layered Image Showcase

```
pah 1
html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Layered Image Showcase</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="showcase">
    <div class="buttons">
      <button id="btn1" class="btn btn-primary">Image 1</button>
      <button id="btn2" class="btn btn-success">Image 2</button>
      <button id="btn3" class="btn btn-danger">Image 3</button>
    </div>

    <div class="image-container">
      <img id="img1" class="layered-img" src="img1.jpg" alt="Image 1">
      <img id="img2" class="layered-img" src="img2.jpg" alt="Image 2">
      <img id="img3" class="layered-img" src="img3.jpg" alt="Image 3">
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>
--------------------------------------------------------------------------------
css
body {
  margin: 0;
  padding: 30px 0;
  font-family: Arial, sans-serif;
  background: #fff;
  text-align: center;
}

.showcase {
  width: 100%;
}

.buttons {
  display: flex;
  justify-content: center;
  gap: 6px;
  margin-bottom: 20px;
}

.buttons button {
  font-size: 12px;
  padding: 5px 12px;
}

.image-container {
  position: relative;
  width: 300px;
```

```
  height: 400px;
  margin: 0 auto;
}

.layered-img {
  position: absolute;
  top: 0;
  left: 0;
  width: 300px;
  height: 400px;
  object-fit: cover;
  border: 1px solid #ddd;
}

#img1 {
  z-index: 1;
}

#img2 {
  z-index: 2;
}

#img3 {
  z-index: 3;
}

@media (max-width: 600px) {
  .image-container {
    width: 250px;
    height: 333px;
  }

  .layered-img {
    width: 250px;
    height: 333px;
  }
}
-----------------------------------------------------------
js
document.addEventListener('DOMContentLoaded', function () {
  const img1 = document.getElementById('img1');
  const img2 = document.getElementById('img2');
  const img3 = document.getElementById('img3');

  const btn1 = document.getElementById('btn1');
  const btn2 = document.getElementById('btn2');
  const btn3 = document.getElementById('btn3');

  function bringToFront(image) {
    img1.style.zIndex = '1';
    img2.style.zIndex = '1';
    img3.style.zIndex = '1';
    image.style.zIndex = '10';
  }

  btn1.addEventListener('click', function () {
    bringToFront(img1);
  });

  btn2.addEventListener('click', function () {
    bringToFront(img2);
  });

  btn3.addEventListener('click', function () {
    bringToFront(img3);
  });
```

```
});
------------------------------------------------------------
```

WEEK 8

Code 5 — Live Textarea Editor

```
5.pah-2
html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Live Textarea Editor</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="container">
    <h2>Live Textarea Editor</h2>

    <div class="controls">
      <div class="control-item">
        <label for="fontSizeSelect">Font Size</label>
        <select id="fontSizeSelect" class="form-select">
          <option value="12px">12px</option>
          <option value="16px">16px</option>
          <option value="20px">20px</option>
        </select>
      </div>

      <div class="control-item">
        <label for="fontColorSelect">Font Color</label>
        <select id="fontColorSelect" class="form-select">
          <option value="black">Black</option>
          <option value="blue">Blue</option>
          <option value="red">Red</option>
        </select>
      </div>

      <div class="btn-group">
        <button id="boldBtn" class="btn btn-outline-dark" type="button">
          Bold
        </button>
        <button id="italicBtn" class="btn btn-outline-dark" type="button">
          Italic
        </button>
        <button id="resetBtn" class="btn btn-outline-secondary" type="button">
          Reset
        </button>
      </div>
    </div>

    <textarea id="editor" placeholder="Type something here..."></textarea>
  </div>

  <script src="script.js"></script>
</body>
</html>
--------------------------------------------------------------------------
css
body {
  margin: 0;
  background: #fff;
```

```
  font-family: Arial, sans-serif;
}

.container {
  max-width: 1100px;
  margin: 35px auto;
  padding: 0 20px;
}

h2 {
  text-align: center;
  font-size: 20px;
  font-weight: 500;
  margin-bottom: 15px;
}

.controls {
  display: flex;
  align-items: end;
  gap: 12px;
  margin-bottom: 8px;
}

.control-item {
  flex: 1;
}

.control-item label {
  display: block;
  font-size: 12px;
  margin-bottom: 5px;
  color: #333;
}

.form-select {
  height: 34px;
  font-size: 12px;
  padding: 5px 10px;
}

.btn-group {
  display: flex;
  height: 34px;
}

.btn-group .btn {
  font-size: 11px;
  padding: 5px 10px;
}

.btn.active {
  background-color: #212529;
  color: white;
}

#editor {
  display: block;
  width: 100%;
  height: 115px;
  resize: vertical;
  box-sizing: border-box;
  padding: 8px;
  border: 1px solid #ddd;
  border-radius: 4px;
  font-size: 12px;
  font-weight: normal;
```

```
  font-style: normal;
  color: black;
  outline: none;
}

#editor:focus {
  border-color: #86b7fe;
  box-shadow: 0 0 0 2px rgba(13, 110, 253, 0.15);
}
--------------------------------------------------------------------------
js
document.addEventListener('DOMContentLoaded', function () {
  const editor = document.getElementById('editor');
  const fontSizeSelect = document.getElementById('fontSizeSelect');
  const fontColorSelect = document.getElementById('fontColorSelect');
  const boldBtn = document.getElementById('boldBtn');
  const italicBtn = document.getElementById('italicBtn');
  const resetBtn = document.getElementById('resetBtn');

  const fontSize = '12px';
  const color = 'black';
  let fontWeight = 'normal';
  let fontStyle = 'normal';

  editor.style.fontSize = fontSize;
  editor.style.color = color;
  editor.style.fontWeight = fontWeight;
  editor.style.fontStyle = fontStyle;

  fontSizeSelect.addEventListener('change', function () {
    editor.style.fontSize = fontSizeSelect.value;
  });

  fontColorSelect.addEventListener('change', function () {
    editor.style.color = fontColorSelect.value;
  });

  boldBtn.addEventListener('click', function () {
    boldBtn.classList.toggle('active');

    if (boldBtn.classList.contains('active')) {
      fontWeight = 'bold';
    } else {
      fontWeight = 'normal';
    }

    editor.style.fontWeight = fontWeight;
  });

  italicBtn.addEventListener('click', function () {
    italicBtn.classList.toggle('active');

    if (italicBtn.classList.contains('active')) {
      fontStyle = 'italic';
    } else {
      fontStyle = 'normal';
    }

    editor.style.fontStyle = fontStyle;
  });

  resetBtn.addEventListener('click', function () {
    fontSizeSelect.value = '12px';
    fontColorSelect.value = 'black';

    boldBtn.classList.remove('active');
```

```
    italicBtn.classList.remove('active');

    fontWeight = 'normal';
    fontStyle = 'normal';

    editor.style.fontSize = '12px';
    editor.style.color = 'black';
    editor.style.fontWeight = 'normal';
    editor.style.fontStyle = 'normal';
  });
});
--------------------------------------------------------------------------------
```

WEEK 9 — PROGRAMS WITH IDENTIFICATION TITLES

Code 1 — Medical Shop Inventory (React)

```
Week 9
Cod 1
src/components/MedicalShop.jsx
import React, { useState } from 'react';

function MedicalShop() {
  const [inventory, setInventory] = useState([
    { id: 1, name: 'Paracetamol', quantity: 50, price: 2.5 },
    { id: 2, name: 'Aspirin', quantity: 30, price: 3 },
    { id: 3, name: 'Band-Aid', quantity: 100, price: 1 }
  ]);

  const [newMedicine, setNewMedicine] = useState({
    name: '',
    quantity: '',
    price: ''
  });

  const handleNewMedicineChange = (event) => {
    const { name, value } = event.target;

    setNewMedicine({
      ...newMedicine,
      [name]: value
    });
  };

  const handleAddMedicine = (event) => {
    event.preventDefault();

    if (
      newMedicine.name.trim() === '' ||
      newMedicine.quantity === '' ||
      newMedicine.price === ''
    ) {
      alert('Please fill in all fields');
      return;
    }

    const newId =
      inventory.length > 0
        ? Math.max(...inventory.map((medicine) => medicine.id)) + 1
        : 1;

    const medicine = {
      id: newId,
      name: newMedicine.name,
      quantity: Number(newMedicine.quantity),
      price: Number(newMedicine.price)
    };

    setInventory([...inventory, medicine]);

    setNewMedicine({
      name: '',
      quantity: '',
      price: ''
    });
  };
```

```
return (
  <div className="container my-4">
    <h1 className="text-primary text-center mb-4">
      Medical Shop Inventory
    </h1>

    <table className="table table-striped table-bordered">
      <thead>
        <tr>
          <th>ID</th>
          <th>Name</th>
          <th>Quantity</th>
          <th>Price</th>
        </tr>
      </thead>

      <tbody>
        {inventory.map((medicine) => (
          <tr key={medicine.id}>
            <td>{medicine.id}</td>
            <td>{medicine.name}</td>
            <td>{medicine.quantity}</td>
            <td>{medicine.price}</td>
          </tr>
        ))}
      </tbody>
    </table>

    <h2 className="text-success text-center mt-4 mb-3">
      Add New Medicine
    </h2>

    <form onSubmit={handleAddMedicine}>
      <div className="row g-2">
        <div className="col-md-4">
          <input
            type="text"
            name="name"
            className="form-control"
            placeholder="Name"
            value={newMedicine.name}
            onChange={handleNewMedicineChange}
          />
        </div>

        <div className="col-md-3">
          <input
            type="number"
            name="quantity"
            className="form-control"
            placeholder="Quantity"
            min="0"
            value={newMedicine.quantity}
            onChange={handleNewMedicineChange}
          />
        </div>

        <div className="col-md-3">
          <input
            type="number"
            name="price"
            className="form-control"
            placeholder="Price"
            min="0"
            step="0.01"
            value={newMedicine.price}
```

```
              onChange={handleNewMedicineChange}
            />
          </div>

          <div className="col-md-2">
            <button
              type="submit"
              className="btn btn-success w-100"
            >
              Add Medicine
            </button>
          </div>
        </div>
      </form>
    </div>
  );
}

export default MedicalShop;
```

---

src/App.js

```
import React from 'react';
import MedicalShop from './components/MedicalShop';
import './App.css';

function App() {
  return <MedicalShop />;
}

export default App;
```

---

src/App.css

```
body {
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', 'Oxygen',
    'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid Sans', 'Helvetica Neue',
    sans-serif;
  background-color: white;
}

.container {
  max-width: 1100px;
}

h1 {
  font-size: 22px;
}

h2 {
  font-size: 20px;
}

.table {
  font-size: 13px;
}

.table th {
  font-weight: 600;
}

.form-control {
  font-size: 12px;
}

.btn {
  font-size: 12px;
```

```
}
-----------------------------------------------------------------
```

WEEK 9

Code 2 — Food Order Management (React)

```
cod 2
src/components/FoodOrder.jsx
import React, { useState } from 'react';

function FoodOrder() {
  const [orders, setOrders] = useState([
    { id: 1, item: 'Burger', quantity: 2, price: 150 },
    { id: 2, item: 'Pizza', quantity: 1, price: 300 },
    { id: 3, item: 'Pasta', quantity: 3, price: 200 }
  ]);

  const [newOrder, setNewOrder] = useState({
    item: '',
    quantity: '',
    price: ''
  });

  const handleNewOrderChange = (event) => {
    const { name, value } = event.target;

    setNewOrder({
      ...newOrder,
      [name]: value
    });
  };

  const handleAddOrder = (event) => {
    event.preventDefault();

    if (
      newOrder.item.trim() === '' ||
      newOrder.quantity === '' ||
      newOrder.price === ''
    ) {
      alert('Please fill in all fields');
      return;
    }

    const newId =
      orders.length > 0
        ? Math.max(...orders.map((order) => order.id)) + 1
        : 1;

    const order = {
      id: newId,
      item: newOrder.item,
      quantity: Number(newOrder.quantity),
      price: Number(newOrder.price)
    };

    setOrders([...orders, order]);

    setNewOrder({
      item: '',
      quantity: '',
      price: ''
    });
  };
```

```
return (
  <div className="container my-4">
    <h1 className="text-primary text-center mb-4">
      Food Order Management
    </h1>

    <table className="table table-striped table-bordered">
      <thead>
        <tr>
          <th>ID</th>
          <th>Food Item</th>
          <th>Quantity</th>
          <th>Price ($)</th>
        </tr>
      </thead>

      <tbody>
        {orders.map((order) => (
          <tr key={order.id}>
            <td>{order.id}</td>
            <td>{order.item}</td>
            <td>{order.quantity}</td>
            <td>{order.price}</td>
          </tr>
        ))}
      </tbody>
    </table>

    <h2 className="text-success text-center mt-4 mb-3">
      Add New Food Order
    </h2>

    <form onSubmit={handleAddOrder}>
      <div className="row g-2">
        <div className="col-md-4">
          <input
            type="text"
            name="item"
            className="form-control"
            placeholder="Food Item"
            value={newOrder.item}
            onChange={handleNewOrderChange}
          />
        </div>

        <div className="col-md-3">
          <input
            type="number"
            name="quantity"
            className="form-control"
            placeholder="Quantity"
            min="1"
            value={newOrder.quantity}
            onChange={handleNewOrderChange}
          />
        </div>

        <div className="col-md-3">
          <input
            type="number"
            name="price"
            className="form-control"
            placeholder="Price ($)"
            min="0"
            step="0.01"
            value={newOrder.price}
```

```
              onChange={handleNewOrderChange}
            />
          </div>

          <div className="col-md-2">
            <button
              type="submit"
              className="btn btn-success w-100"
            >
              Add Order
            </button>
          </div>
        </div>
      </form>
    </div>
  );
}

export default FoodOrder;
```

---

src/App.js

```
import React from 'react';
import FoodOrder from './components/FoodOrder';
import './App.css';

function App() {
  return <FoodOrder />;
}

export default App;
```

---

src/App.css

```
body {
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', 'Oxygen',
    'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid Sans', 'Helvetica Neue',
    sans-serif;
  background-color: white;
}

.container {
  max-width: 1100px;
}

h1 {
  font-size: 22px;
}

h2 {
  font-size: 20px;
}

.table {
  font-size: 13px;
}

.table th {
  font-weight: 600;
}

.form-control {
  font-size: 12px;
}

.btn {
  font-size: 12px;
```

```
}
------------------------------------------------------------
```

WEEK 9

Code 3 — Music Store Inventory (React)

1.cy
src/components/MusicStore.jsx

```
import React, { useState } from 'react';

function MusicStore() {
  const [inventory, setInventory] = useState([
    { id: 1, name: 'Acoustic Guitar', quantity: 10, price: 150 },
    { id: 2, name: 'Electric Keyboard', quantity: 5, price: 300 },
    { id: 3, name: 'Drum Set', quantity: 3, price: 500 }
  ]);

  const [newItem, setNewItem] = useState({
    name: '',
    quantity: '',
    price: ''
  });

  const handleNewItemChange = (event) => {
    const { name, value } = event.target;

    setNewItem({
      ...newItem,
      [name]: value
    });
  };

  const handleAddItem = (event) => {
    event.preventDefault();

    if (
      newItem.name.trim() === '' ||
      newItem.quantity === '' ||
      newItem.price === ''
    ) {
      alert('Please fill in all fields');
      return;
    }

    const newId =
      inventory.length > 0
        ? Math.max(...inventory.map((item) => item.id)) + 1
        : 1;

    const item = {
      id: newId,
      name: newItem.name,
      quantity: Number(newItem.quantity),
      price: Number(newItem.price)
    };

    setInventory([...inventory, item]);

    setNewItem({
      name: '',
      quantity: '',
      price: ''
    });
  };
```

```
return (
  <div className="container my-4">
    <h1 className="bg-dark text-warning text-center fw-bold py-3 rounded">
      Music Store Inventory
    </h1>

    <table className="table table-bordered table-hover shadow-sm">
      <thead className="table-warning">
        <tr>
          <th>ID</th>
          <th>Instrument / Album</th>
          <th>Quantity</th>
          <th>Price ($)</th>
        </tr>
      </thead>

      <tbody>
        {inventory.map((item) => (
          <tr key={item.id}>
            <td>
              <span className="badge text-dark">
                {item.id}
              </span>
            </td>
            <td className="fw-bold">{item.name}</td>
            <td>{item.quantity}</td>
            <td className="text-success">
              ${item.price.toFixed(2)}
            </td>
          </tr>
        ))}
      </tbody>
    </table>

    <h2 className="text-info text-center mt-4 mb-3">
      Add New Instrument
    </h2>

    <form onSubmit={handleAddItem}>
      <div className="row g-2">
        <div className="col-md-4">
          <input
            type="text"
            name="name"
            className="form-control border-info"
            placeholder="Instrument / Album Name"
            value={newItem.name}
            onChange={handleNewItemChange}
          />
        </div>

        <div className="col-md-3">
          <input
            type="number"
            name="quantity"
            className="form-control border-info"
            placeholder="Quantity"
            min="0"
            value={newItem.quantity}
            onChange={handleNewItemChange}
          />
        </div>

        <div className="col-md-3">
          <input
            type="number"
```

```
              name="price"
              className="form-control border-info"
              placeholder="Price ($)"
              min="0"
              step="0.01"
              value={newItem.price}
              onChange={handleNewItemChange}
            />
          </div>

          <div className="col-md-2">
            <button
              type="submit"
              className="btn btn-warning w-100 fw-bold"
            >
              Add Item
            </button>
          </div>
        </div>
      </form>
    </div>
  );
}

export default MusicStore;
---------------------------------------------------------------------------
src/App.js
import React from 'react';
import MusicStore from './components/MusicStore';
import './App.css';

function App() {
  return <MusicStore />;
}

export default App;
===========================================================
src/App.css
body {
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', 'Oxygen',
    'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid Sans', 'Helvetica Neue',
    sans-serif;
  background-color: white;
}

.container {
  max-width: 1100px;
}

h1 {
  font-size: 22px;
}

h2 {
  font-size: 20px;
}

.table {
  font-size: 13px;
}

.table th {
  font-weight: 600;
}
```

```
.form-control {
  font-size: 12px;
}

.btn {
  font-size: 12px;
}
```

WEEK 9

Code 4 — Bookstore Inventory (React)

pah 1
src/components/Bookstore.jsx

```
import React, { useState } from 'react';

function Bookstore() {
  const [inventory, setInventory] = useState([
    { id: 1, title: 'The Great Gatsby', author: 'F. Scott Fitzgerald', stock: 12, price: 10.5 },
    { id: 2, title: '1984', author: 'George Orwell', stock: 8, price: 9.0 },
    { id: 3, title: 'To Kill a Mockingbird', author: 'Harper Lee', stock: 15, price: 12.0 }
  ]);

  const [newBook, setNewBook] = useState({
    title: '',
    author: '',
    stock: '',
    price: ''
  });

  const handleNewBookChange = (event) => {
    const { name, value } = event.target;

    setNewBook({
      ...newBook,
      [name]: value
    });
  };

  const handleAddBook = (event) => {
    event.preventDefault();

    if (
      newBook.title.trim() === '' ||
      newBook.author.trim() === '' ||
      newBook.stock === '' ||
      newBook.price === ''
    ) {
      alert('Please fill in all fields');
      return;
    }

    const newId = Math.max(...inventory.map(book => book.id)) + 1;

    const book = {
      id: newId,
      title: newBook.title,
      author: newBook.author,
      stock: Number(newBook.stock),
      price: Number(newBook.price)
    };

    setInventory([...inventory, book]);

    setNewBook({
      title: '',
      author: '',
      stock: '',
      price: ''
    });
  };
```

```
return (
  <div className="container my-5">
    <h1 className="text-primary text-center fw-bold">
      Bookstore Inventory
    </h1>

    <p className="text-muted text-center">
      Manage your book collection with ease
    </p>

    <h2 className="text-success mt-4 mb-3">
      Available Books
    </h2>

    <div className="row">
      {inventory.map((book) => (
        <div className="col-md-4 mb-3" key={book.id}>
          <div className="card h-100 shadow-sm">
            <div className="card-body">
              <h5 className="card-title text-primary fw-bold">
                {book.title}
              </h5>

              <p className="card-text mb-1">
                by {book.author}
              </p>

              <p className="card-text mb-1">
                <strong>Stock:</strong> {book.stock}
              </p>

              <p className="card-text mb-0">
                <strong>Price:</strong> ${book.price.toFixed(2)}
              </p>
            </div>

            <div className="card-footer text-center">
              <span className="badge bg-info">
                ID: {book.id}
              </span>
            </div>
          </div>
        </div>
      ))}
    </div>

    <h2 className="text-success mt-4 mb-3">
      Add a New Book
    </h2>

    <div className="card shadow-sm p-2">
      <form onSubmit={handleAddBook}>
        <div className="row g-2">
          <div className="col-md-3">
            <input
              type="text"
              name="title"
              className="form-control"
              placeholder="Book Title"
              value={newBook.title}
              onChange={handleNewBookChange}
            />
          </div>

          <div className="col-md-3">
            <input
```

```
                type="text"
                name="author"
                className="form-control"
                placeholder="Author"
                value={newBook.author}
                onChange={handleNewBookChange}
              />
            </div>

            <div className="col-md-2">
              <input
                type="number"
                name="stock"
                className="form-control"
                placeholder="Stock"
                min="0"
                value={newBook.stock}
                onChange={handleNewBookChange}
              />
            </div>

            <div className="col-md-2">
              <input
                type="number"
                name="price"
                className="form-control"
                placeholder="Price ($)"
                min="0"
                step="0.01"
                value={newBook.price}
                onChange={handleNewBookChange}
              />
            </div>

            <div className="col-md-2">
              <button
                type="submit"
                className="btn btn-success w-100"
              >
                Add Book
              </button>
            </div>
          </div>
        </form>
      </div>
    </div>
  );
}

export default Bookstore;
================================================================
src/App.js
import React from 'react';
import Bookstore from './components/Bookstore';
import './App.css';

function App() {
  return <Bookstore />;
}

export default App;
================================================================
src/App.css
body {
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', 'Oxygen',
```

```
    'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid Sans', 'Helvetica Neue',
    sans-serif;
  background-color: white;
}

.container {
  max-width: 1100px;
}

h1 {
  font-size: 22px;
}

h2 {
  font-size: 18px;
}

.card {
  font-size: 12px;
}

.card-title {
  font-size: 13px;
}

.card-footer {
  padding: 4px;
}

.form-control {
  font-size: 11px;
}

.btn {
  font-size: 11px;
}

.badge {
  font-size: 9px;
}
================================================================================
```

WEEK 10 — PROGRAMS WITH IDENTIFICATION TITLES

Code 1 — Employee Directory (React Class Component)

```
Week 10
Cod-1
src/App.js
import React from 'react';
import './App.css';

class App extends React.Component {
  constructor(props) {
    super(props);

    this.state = {
      query: '',
      employees: [
        'John Doe',
        'Jane Smith',
        'Mike Johnson',
        'Emily Davis',
        'James Brown',
        'Jennifer Wilson',
        'Paul Garcia',
        'Laura Martinez'
      ],
      filteredEmployees: [
        'John Doe',
        'Jane Smith',
        'Mike Johnson',
        'Emily Davis',
        'James Brown',
        'Jennifer Wilson',
        'Paul Garcia',
        'Laura Martinez'
      ]
    };
  }

  componentDidUpdate(prevProps, prevState) {
    if (prevState.query !== this.state.query) {
      this.filterEmployees();
    }
  }

  handleSearch = (event) => {
    this.setState({
      query: event.target.value
    });
  };

  filterEmployees = () => {
    const filteredEmployees = this.state.employees.filter((employee) =>
      employee.toLowerCase().includes(this.state.query.toLowerCase())
    );

    if (
      JSON.stringify(filteredEmployees) !==
      JSON.stringify(this.state.filteredEmployees)
    ) {
      this.setState({
        filteredEmployees: filteredEmployees
      });
```

```
    }
  };

  render() {
    return (
      <div className="App">
        <h1>Employee Directory</h1>

        <input
          type="text"
          placeholder="Search employees..."
          value={this.state.query}
          onChange={this.handleSearch}
        />

        <div className="employee-list">
          {this.state.filteredEmployees.map((employee, index) => (
            <div className="employee" key={index}>
              {employee}
            </div>
          ))}
        </div>
      </div>
    );
  }
}

export default App;
```

```
================================================================================
src/App.css
```

```
.App {
  text-align: center;
  min-height: 100vh;
  background-color: #f5f5f5;
  padding-top: 15px;
}

.App h1 {
  font-size: 28px;
  color: #333;
  margin-bottom: 32px;
}

.App input {
  width: 325px;
  height: 32px;
  padding: 0 10px;
  border: 1px solid #ccc;
  border-radius: 3px;
  font-size: 12px;
  box-sizing: border-box;
  outline: none;
}

.App input:focus {
  border-color: #66afe9;
}

.employee-list {
  width: 310px;
  margin: 16px auto 0;
}

.employee {
  background-color: white;
  border: 1px solid #ddd;
```

```
  border-radius: 3px;
  height: 32px;
  line-height: 32px;
  margin-bottom: 8px;
  color: #333;
  font-size: 14px;
}
============================================================
src/index.js
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App';
import './index.css';

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById('root')
);
============================================================
src/index.css
body {
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', 'Oxygen',
    'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid Sans', 'Helvetica Neue',
    sans-serif;
}
================================================================
```

WEEK 10

Code 2 — Stopwatch (React Class Component)

2.cod
src/components/Stopwatch.jsx

```
import React from 'react';

class Stopwatch extends React.Component {
  constructor(props) {
    super(props);

    this.state = {
      time: 0,
      running: false,
      laps: []
    };

    this.timer = null;
    this.startTime = 0;
    this.elapsedBeforeStart = 0;
  }

  componentDidMount() {
    this.timer = null;
  }

  componentDidUpdate(prevProps, prevState) {
    if (!prevState.running && this.state.running) {
      this.startTimer();
    }

    if (prevState.running && !this.state.running) {
      this.stopTimer();
    }
  }

  componentWillUnmount() {
    this.stopTimer();
  }

  startTimer = () => {
    this.startTime = Date.now() - this.elapsedBeforeStart;

    this.timer = setInterval(() => {
      this.setState({
        time: Date.now() - this.startTime
      });
    }, 10);
  };

  stopTimer = () => {
    if (this.timer) {
      clearInterval(this.timer);
      this.timer = null;
    }

    this.elapsedBeforeStart = this.state.time;
  };

  start = () => {
    if (!this.state.running) {
```

```
    this.setState({
      running: true
    });
  }
};

stop = () => {
  if (this.state.running) {
    const currentTime = this.state.time;

    this.setState({
      running: false,
      laps: [...this.state.laps, currentTime]
    });
  }
};

reset = () => {
  this.stopTimer();

  this.setState({
    time: 0,
    running: false,
    laps: []
  });

  this.startTime = 0;
  this.elapsedBeforeStart = 0;
};

addLap = () => {
  if (this.state.running) {
    this.setState({
      laps: [...this.state.laps, this.state.time]
    });
  }
};

formatTime = (milliseconds) => {
  const minutes = Math.floor(milliseconds / 60000);
  const seconds = Math.floor((milliseconds % 60000) / 1000);
  const centiseconds = Math.floor((milliseconds % 1000) / 10);

  return (
    String(minutes).padStart(2, '0') +
    ':' +
    String(seconds).padStart(2, '0') +
    '.' +
    String(centiseconds).padStart(2, '0')
  );
};

render() {
  const { time, running, laps } = this.state;

  return (
    <div className="stopwatch">
      <h2>Stopwatch</h2>

      <div className="time">
        {this.formatTime(time)}
      </div>

      <div className="buttons">
        <button
          className="start-btn"
```

```
            onClick={this.start}
            disabled={running}
          >
            Start
          </button>

          <button
            className="stop-btn"
            onClick={this.stop}
            disabled={!running}
          >
            Stop
          </button>

          <button
            className="reset-btn"
            onClick={this.reset}
            disabled={running}
          >
            Reset
          </button>
        </div>

        <button
          className="lap-btn"
          onClick={this.addLap}
          disabled={!running}
        >
          Add Lap
        </button>

        <div className="laps">
          {laps.map((lap, index) => (
            <div className="lap" key={index}>
              {this.formatTime(lap)}
            </div>
          ))}
        </div>
      </div>
    );
  }
}

export default Stopwatch;
=====================================================
src/App.js
import React from 'react';
import Stopwatch from './components/Stopwatch';
import './App.css';

function App() {
  return (
    <div className="App">
      <Stopwatch />
    </div>
  );
}

export default App;
=========================================================
src/App.css
.App {
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: flex-start;
```

```
  padding-top: 110px;
  background-color: #fff;
  font-family: Arial, sans-serif;
}

.stopwatch {
  width: 205px;
  min-height: 150px;
  border: 1px solid #555;
  border-radius: 4px;
  box-shadow: 0 1px 4px rgba(0, 0, 0, 0.35);
  background-color: #fff;
  text-align: center;
  padding: 18px 15px 15px;
  box-sizing: border-box;
}

.stopwatch h2 {
  font-size: 13px;
  margin: 5px 0 8px;
  font-weight: bold;
  color: #333;
}

.time {
  font-size: 20px;
  color: #333;
  margin-bottom: 12px;
}

.buttons {
  display: flex;
  justify-content: center;
  gap: 7px;
}

button {
  border: none;
  border-radius: 2px;
  color: white;
  font-size: 9px;
  padding: 5px 11px;
  cursor: pointer;
}

button:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

.start-btn {
  background-color: #28a745;
}

.stop-btn {
  background-color: #dc3545;
}

.reset-btn {
  background-color: #17a2b8;
}

.lap-btn {
  background-color: #ffc107;
  color: #222;
  margin-top: 7px;
```

```
}

.laps {
  margin-top: 7px;
}

.lap {
  font-size: 9px;
  color: #777;
  margin-top: 3px;
}
==================================================
src/index.js
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App';

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById('root')
);
==================================================
src/index.css
body {
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', 'Oxygen',
    'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid Sans', 'Helvetica Neue',
    sans-serif;
}
==================================================
```

WEEK 10

Code 3 — Crop Dashboard (React)

```
cy
src/Components/CropDashboard.jsx
import React, { useState } from 'react';
import CropList from './CropList';

function CropDashboard({ initialCrops }) {
  const [crops] = useState(initialCrops);
  const [searchTerm, setSearchTerm] = useState('');

  const filteredCrops = crops.filter((crop) =>
    crop.name.toLowerCase().includes(searchTerm.toLowerCase())
  );

  return (
    <div className="dashboard">
      <h1>CROP MANAGEMENT SYSTEM</h1>

      <label htmlFor="cropSearch">Search by Crop Name:</label>

      <input
        id="cropSearch"
        type="text"
        placeholder="Search crops..."
        value={searchTerm}
        onChange={(event) => setSearchTerm(event.target.value)}
      />

      <CropList crops={filteredCrops} />
    </div>
  );
}

export default CropDashboard;
============================================================
src/Components/CropList.jsx
import React from 'react';

function CropList({ crops }) {
  return (
    <div className="crop-list">
      {crops.map((crop, index) => (
        <div className="crop-item" key={index}>
          <strong>{crop.name}</strong>
          <span>- Type: {crop.type} - Status: {crop.status}</span>
        </div>
      ))}
    </div>
  );
}

export default CropList;
=========================================================
src/App.js
import React from 'react';
import CropDashboard from './Components/CropDashboard';
import './App.css';

function App() {
  const initialCrops = [
```

```
    { name: 'Tomato', type: 'Vegetables', status: 'Growing' },
    { name: 'Apple', type: 'Fruits', status: 'Planted' },
    { name: 'Wheat', type: 'Grains', status: 'Harvested' },
    { name: 'Carrot', type: 'Vegetables', status: 'Growing' },
    { name: 'Banana', type: 'Fruits', status: 'Growing' }
  ];

  return <CropDashboard initialCrops={initialCrops} />;
}

export default App;
============================================================
src/App.css
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f1f4f7;
}

.App {
  min-height: 100vh;
}

.dashboard {
  width: 394px;
  margin: 45px auto;
  padding: 38px 24px 30px;
  background: white;
  border-radius: 10px;
  box-shadow: 0 5px 15px rgba(0, 0, 0, 0.1);
  box-sizing: border-box;
}

.dashboard h1 {
  margin: 0 0 25px;
  padding-bottom: 7px;
  border-bottom: 3px solid #2ecc71;
  color: #30465b;
  font-size: 24px;
  letter-spacing: 1px;
}

.dashboard label {
  display: block;
  margin-bottom: 6px;
  color: #30465b;
  font-size: 11px;
  font-weight: bold;
}

.dashboard input {
  width: 100%;
  height: 31px;
  padding: 6px 9px;
  box-sizing: border-box;
  border: 1px solid #ddd;
  border-radius: 5px;
  font-size: 11px;
  font-style: italic;
  outline: none;
  margin-bottom: 20px;
}

.dashboard input:focus {
  border-color: #2ecc71;
}
```

```
.crop-list {
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.crop-item {
  min-height: 30px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0 10px;
  box-sizing: border-box;
  background: #f7f7f7;
  border-left: 3px solid #2ecc71;
  border-radius: 6px;
  color: #222;
  font-size: 11px;
}

.crop-item strong {
  color: #30465b;
  font-size: 11px;
}

.crop-item span {
  text-align: right;
}
```

```
=======================================================
src/index.js
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App';
import './index.css';

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById('root')
);
=============================================================
src/index.css
body {
  margin: 0;
}
=======================================================
```

WEEK 10

Code 4 — Task Management Dashboard (React)

```
pah
1. src/App.js
import React, { Component } from 'react';
import './App.css';

class App extends Component {
  constructor(props) {
    super(props);
    this.state = {
      tasks: [
        { id: 1, title: 'Complete React Assignment', priority: 'High', completed: false, createdAt: Date.now() },
        { id: 2, title: 'Review Bootstrap Documentation', priority: 'Medium', completed: true, createdAt: Date.now() },
        { id: 3, title: 'Debug Lifecycle Methods', priority: 'High', completed: false, createdAt: Date.now() },
        { id: 4, title: 'Setup React Dev Tools', priority: 'Low', completed: false, createdAt: Date.now() }
      ],
      filter: 'All',
      isLoading: true,
      mountTime: null,
      showModal: false,
      newTaskTitle: '',
      newTaskPriority: 'High'
    };

    this.handleFilterChange = this.handleFilterChange.bind(this);
    this.handleTaskToggle = this.handleTaskToggle.bind(this);
    this.handleAddTask = this.handleAddTask.bind(this);
    this.handleDeleteTask = this.handleDeleteTask.bind(this);
    this.handleShowModal = this.handleShowModal.bind(this);
    this.handleCloseModal = this.handleCloseModal.bind(this);
    this.handleNewTaskTitleChange = this.handleNewTaskTitleChange.bind(this);
    this.handleNewTaskPriorityChange = this.handleNewTaskPriorityChange.bind(this);
    this.handleSubmitNewTask = this.handleSubmitNewTask.bind(this);
  }

  componentDidMount() {
    const mountTime = Date.now();
    this.setState({ mountTime });
    console.log('Component mounted at:', mountTime);

    this.loadingTimer = setTimeout(() => {
      this.setState({ isLoading: false });
      console.log('Simulated API call complete');
    }, 1500);
  }

  componentDidUpdate(prevProps, prevState) {
    if (prevState.filter !== this.state.filter) {
      console.log('Filter changed from', prevState.filter, 'to', this.state.filter);
    }
  }

  componentWillUnmount() {
    if (this.loadingTimer) {
      clearTimeout(this.loadingTimer);
    }
    console.log('Component unmounted at:', Date.now());
  }

  handleFilterChange(filterType) {
```

```
  this.setState({ filter: filterType });
}

handleTaskToggle(taskId) {
  this.setState(prevState => ({
    tasks: prevState.tasks.map(task =>
      task.id === taskId ? { ...task, completed: !task.completed } : task
    )
  }));
}

handleAddTask(taskData) {
  const newTask = {
    id: Date.now(),
    title: taskData.title,
    priority: taskData.priority,
    completed: false,
    createdAt: Date.now()
  };
  this.setState(prevState => ({
    tasks: [...prevState.tasks, newTask]
  }));
}

handleDeleteTask(taskId) {
  this.setState(prevState => ({
    tasks: prevState.tasks.filter(task => task.id !== taskId)
  }));
}

handleShowModal() {
  this.setState({ showModal: true });
}

handleCloseModal() {
  this.setState({ showModal: false, newTaskTitle: '', newTaskPriority: 'High' });
}

handleNewTaskTitleChange(e) {
  this.setState({ newTaskTitle: e.target.value });
}

handleNewTaskPriorityChange(e) {
  this.setState({ newTaskPriority: e.target.value });
}

handleSubmitNewTask(e) {
  e.preventDefault();
  if (this.state.newTaskTitle.trim() === '') {
    return;
  }
  this.handleAddTask({
    title: this.state.newTaskTitle,
    priority: this.state.newTaskPriority
  });
  this.handleCloseModal();
}

getFilteredTasks() {
  const { tasks, filter } = this.state;
  if (filter === 'All') {
    return tasks;
  }
  if (filter === 'Completed') {
    return tasks.filter(task => task.completed);
  }
```

```
  return tasks.filter(task => task.priority === filter);
}

getBadgeClass(priority) {
  if (priority === 'High') {
    return 'bg-primary';
  }
  if (priority === 'Medium') {
    return 'bg-warning text-dark';
  }
  return 'bg-secondary';
}

render() {
  const { isLoading, filter, showModal, newTaskTitle, newTaskPriority } = this.state;
  const filteredTasks = this.getFilteredTasks();
  const filters = ['All', 'High', 'Medium', 'Low', 'Completed'];

  return (
    <div className="App">
      <nav className="navbar navbar-dark bg-dark px-3">
        <span className="navbar-brand mb-0 h1">Task Manager</span>
        <button className="btn btn-primary" onClick={this.handleShowModal}>
          Add New Task
        </button>
      </nav>

      <div className="container mt-3">
        <div className="btn-group mb-3" role="group">
          {filters.map(f => (
            <button
              key={f}
              type="button"
              className={`btn ${filter === f ? 'btn-primary' : 'btn-outline-primary'}`}
              onClick={() => this.handleFilterChange(f)}
            >
              {f}
            </button>
          ))}
        </div>

        {isLoading && (
          <div className="d-flex justify-content-center mt-5 mb-3">
            <div className="spinner-border text-primary" role="status">
              <span className="visually-hidden">Loading...</span>
            </div>
          </div>
        )}

        <div className="row">
          {filteredTasks.length === 0 ? (
            <div className="col-12">
              <div className="alert alert-info" role="alert">
                No tasks found for this filter.
              </div>
            </div>
          ) : (
            filteredTasks.map(task => (
              <div className="col-md-6 mb-3" key={task.id}>
                <div className={`card ${task.completed ? 'bg-light border-success' : ''}`}>
                  <div className="card-body d-flex justify-content-between align-items-center">
                    <div className="form-check">
                      <input
                        className="form-check-input"
                        type="checkbox"
                        id={`task-checkbox-${task.id}`}
```

```
                  checked={task.completed}
                  onChange={() => this.handleTaskToggle(task.id)}
                />
                <label
                  className="form-check-label"
                  htmlFor={`task-checkbox-${task.id}`}
                  style={{ textDecoration: task.completed ? 'line-through' : 'none' }}
                >
                  {task.title}
                </label>
                <div>
                  <button
                    className="btn btn-danger btn-sm mt-2"
                    onClick={() => this.handleDeleteTask(task.id)}
                  >
                    Delete
                  </button>
                </div>
              </div>
              <span className={`badge ${this.getBadgeClass(task.priority)}`}>
                {task.priority}
              </span>
            </div>
          </div>
        </div>
      ))
    )}
  </div>
</div>

{showModal && (
  <div className="modal d-block" tabIndex="-1" style={{ backgroundColor: 'rgba(0,0,0,0.5)' }}>
    <div className="modal-dialog">
      <div className="modal-content">
        <div className="modal-header">
          <h5 className="modal-title">Add New Task</h5>
          <button type="button" className="btn-close" onClick={this.handleCloseModal}></button>
        </div>
        <form onSubmit={this.handleSubmitNewTask}>
          <div className="modal-body">
            <div className="mb-3">
              <label className="form-label" htmlFor="new-task-title">Task Title</label>
              <input
                type="text"
                id="new-task-title"
                className="form-control"
                placeholder="Enter task title..."
                value={newTaskTitle}
                onChange={this.handleNewTaskTitleChange}
              />
            </div>
            <div className="mb-3">
              <label className="form-label" htmlFor="new-task-priority">Priority</label>
              <select
                id="new-task-priority"
                className="form-select"
                value={newTaskPriority}
                onChange={this.handleNewTaskPriorityChange}
              >
                <option value="High">High</option>
                <option value="Medium">Medium</option>
                <option value="Low">Low</option>
              </select>
            </div>
          </div>
          <div className="modal-footer">
```

```
                  <button type="button" className="btn btn-secondary" onClick={this.handleCloseModal}>
                    Close
                  </button>
                  <button type="submit" className="btn btn-primary">
                    Add Task
                  </button>
                </div>
              </form>
            </div>
          </div>
        </div>
      )}
    </div>
  );
 }
}

export default App;
```

```
2. src/App.css
.App {
  min-height: 100vh;
  background-color: #f8f9fa;
}

.card {
  transition: all 0.2s ease-in-out;
}

.form-check-label {
  margin-left: 8px;
}
```

```
3. src/index.js
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
import reportWebVitals from './reportWebVitals';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

reportWebVitals();
```

```
4. src/index.css
body {
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, 'Helvetica Neue', Arial, sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

code {
  font-family: source-code-pro, Menlo, Monaco, Consolas, 'Courier New', monospace;
}
```

```
5. src/setupTests.js
import '@testing-library/jest-dom';
```

WEEK 11 — PROGRAMS WITH IDENTIFICATION TITLES

Code 1 — Book Card (React Props and PropTypes)

```
Week 11 COD

1. src/components/BookCard.jsx
import React from "react";
import PropTypes from "prop-types";
import "./BookCard.css";

function BookCard({ bookTitle, authorName, availabilityStatus }) {
  return (
    <div id="book-card">
      <h2 id="book-title">{bookTitle}</h2>
      <p id="author-name">Author: {authorName}</p>
      <p id="availability-status">Status: {availabilityStatus}</p>
    </div>
  );
}

BookCard.propTypes = {
  bookTitle: PropTypes.string.isRequired,
  authorName: PropTypes.string.isRequired,
  availabilityStatus: PropTypes.string.isRequired,
};

export default BookCard;
================================================================================

2. src/components/BookCard.css

#book-card {
  background: white;
  border: 2px solid #6c63ff;
  border-radius: 10px;
  padding: 24px;
  max-width: 400px;
  margin: 40px auto;
  text-align: center;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}

#book-title {
  color: #6c63ff;
  font-size: 24px;
  margin-bottom: 12px;
}

#author-name {
  color: #555;
  font-size: 16px;
  margin-bottom: 8px;
}

#availability-status {
  color: #27ae60;
  font-size: 18px;
  font-weight: bold;
}
================================================================================

3. src/App.js
```

```
import React from "react";
import BookCard from "./components/BookCard";

function App() {
  return (
    <div>
      <BookCard
        bookTitle="The Alchemist"
        authorName="Paulo Coelho"
        availabilityStatus="Available"
      />
    </div>
  );
}

export default App;
```

==========

4. src/App.css

Delete everything inside App.css and leave it as:

==========

5. src/index.css

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #f1f5f9;
}
```

==========

6. src/index.js

```
import React from "react";
import ReactDOM from "react-dom";
import App from "./App";

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById("root")
);
```

==========

WEEK 11

Code 2 — Price Tag with Discount (React State)

```
2.COD

1. src/components/PriceTag.jsx
import React, { useState } from "react";
import PropTypes from "prop-types";
import "./PriceTag.css";

function PriceTag({ productName, originalPrice }) {
  const [isDiscounted, setIsDiscounted] = useState(false);

  const displayedPrice = isDiscounted
    ? originalPrice * 0.9
    : originalPrice;

  const toggleDiscount = () => {
    setIsDiscounted(!isDiscounted);
  };

  return (
    <div id="price-tag">
      <h2 id="product-name">{productName}</h2>

      <p id="displayed-price">
        Price: ■{displayedPrice}
      </p>

      <button id="toggle-discount" onClick={toggleDiscount}>
        {isDiscounted ? "Remove Discount" : "Apply Discount"}
      </button>
    </div>
  );
}

PriceTag.propTypes = {
  productName: PropTypes.string.isRequired,
  originalPrice: PropTypes.number.isRequired,
};

export default PriceTag;
================================================================================

2. src/components/PriceTag.css
#price-tag {
  background: white;
  border: 2px solid #e67e22;
  border-radius: 10px;
  padding: 24px;
  max-width: 400px;
  margin: 40px auto;
  text-align: center;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}

#product-name {
  color: #e67e22;
  font-size: 24px;
  margin-bottom: 12px;
}
```

```
#displayed-price {
  color: #2c3e50;
  font-size: 20px;
  margin-bottom: 16px;
}

#toggle-discount {
  background: #e67e22;
  color: white;
  border: none;
  padding: 10px 20px;
  border-radius: 6px;
  font-size: 16px;
  cursor: pointer;
}
```

==========

3. src/App.js

```
import React from "react";
import PriceTag from "./components/PriceTag";

function App() {
  return (
    <div>
      <PriceTag
        productName="Wireless Headphones"
        originalPrice={2000}
      />
    </div>
  );
}

export default App;
```

==========

4. src/App.css

You don't need anything here. You can leave it empty.

==========

5. src/index.css

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #f1f5f9;
}
```

==========

6. src/index.js

```
import React from "react";
import ReactDOM from "react-dom";
import App from "./App";
import "./index.css";

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById("root")
);
```

==========

WEEK 11

Code 3 — Student Dashboard and Marks (React Router)

```
Week 11 CY
1. src/components/Navbar.jsx
import React from "react";
import { Link } from "react-router-dom";
import PropTypes from "prop-types";
import "./Navbar.css";

function Navbar({ studentName }) {
  return (
    <nav id="navbar">
      <span id="nav-brand">Student Dashboard</span>

      <div id="nav-links">
        <Link to="/" id="home-link">
          Home
        </Link>

        <Link to="/marks" id="marks-link">
          Marks
        </Link>
      </div>

      <span id="nav-student">Welcome, {studentName}</span>
    </nav>
  );
}

Navbar.propTypes = {
  studentName: PropTypes.string.isRequired,
};

export default Navbar;
================================================================================

2. src/components/Navbar.css
#navbar {
  background: #1a237e;
  padding: 14px 28px;
  display: flex;
  align-items: center;
  justify-content: space-between;
}

#nav-brand {
  color: white;
  font-size: 20px;
  font-weight: bold;
}

#nav-links {
  display: flex;
  gap: 20px;
}

#nav-links a {
  color: white;
  text-decoration: none;
  font-size: 16px;
}
```

```
#nav-student {
  color: #90caf9;
  font-size: 15px;
}
================================================================================

3. src/components/Marks.jsx
import React, { useState, useEffect } from "react";
import PropTypes from "prop-types";
import "./Marks.css";

function Marks({ marks }) {
  const [totalScore, setTotalScore] = useState(0);

  useEffect(() => {
    const total = marks.reduce((sum, mark) => sum + mark.score, 0);
    setTotalScore(total);

    console.log("Visited Marks Page");
  }, [marks]);

  return (
    <div id="marks-container">
      <h1 id="marks-title">Subject Marks</h1>

      <div id="marks-list">
        {marks.map((mark, index) => (
          <div className="mark-item" key={index}>
            <span className="mark-subject">{mark.subject}</span>
            <span className="mark-score">Score: {mark.score}</span>
          </div>
        ))}
      </div>

      <p id="total-score">Total Score: {totalScore}</p>
    </div>
  );
}

Marks.propTypes = {
  marks: PropTypes.arrayOf(
    PropTypes.shape({
      subject: PropTypes.string,
      score: PropTypes.number,
    })
  ).isRequired,
};

export default Marks;
================================================================================

4. src/components/Marks.css
#marks-container {
  max-width: 600px;
  margin: 60px auto;
  background: white;
  padding: 32px;
  border-radius: 10px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}

#marks-title {
  color: #1a237e;
  font-size: 26px;
  margin-bottom: 20px;
  text-align: center;
```

```
}

.mark-item {
  display: flex;
  justify-content: space-between;
  padding: 10px 0;
  border-bottom: 1px solid #eee;
}

.mark-subject {
  color: #333;
  font-size: 16px;
}

.mark-score {
  color: #27ae60;
  font-size: 16px;
  font-weight: bold;
}

#total-score {
  color: #1a237e;
  font-size: 18px;
  font-weight: bold;
  margin-top: 16px;
  text-align: right;
}
================================================================================

5. src/App.js
import React from "react";
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Navbar from "./components/Navbar";
import Marks from "./components/Marks";
import "./App.css";

function App() {
  const marks = [
    { subject: "Mathematics", score: 92 },
    { subject: "Physics", score: 85 },
    { subject: "Chemistry", score: 78 },
    { subject: "Computer Science", score: 96 },
    { subject: "English", score: 88 },
  ];

  return (
    <BrowserRouter>
      <div id="app-container">
        <Navbar studentName="Meena Sharma" />

        <Routes>
          <Route
            path="/"
            element={
              <div id="home-container">
                <h1 id="home-title">Welcome to Your Dashboard</h1>
                <p id="home-student-name">Student: Meena Sharma</p>
                <p id="home-department">
                  Department: Computer Science
                </p>
                <p id="home-message">
                  Use the navigation bar to explore your Marks.
                </p>
              </div>
            }
          />
```

```
          <Route path="/marks" element={<Marks marks={marks} />} />
        </Routes>
      </div>
    </BrowserRouter>
  );
}

export default App;
```

==========

6. src/App.css

```
#app-container {
  font-family: Arial, sans-serif;
  min-height: 100vh;
  background: #f0f4f8;
}

#home-container {
  max-width: 600px;
  margin: 60px auto;
  background: white;
  padding: 32px;
  border-radius: 10px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
  text-align: center;
}

#home-title {
  color: #1a237e;
  font-size: 26px;
  margin-bottom: 16px;
}

#home-student-name,
#home-department {
  color: #333;
  font-size: 16px;
  margin-bottom: 8px;
}

#home-message {
  color: #888;
  font-size: 14px;
  margin-top: 16px;
}
```

==========

7. src/index.css

```
body {
  margin: 0;
  padding: 0;
}
```

==========

8. src/index.js

```
import React from "react";
import ReactDOM from "react-dom";
import App from "./App";

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById("root")
);
```

==========

WEEK 11

Code 4 — Employee Card (React Props)

```
1. PAH
1. src/components/EmployeeCard.jsx
import React from "react";
import PropTypes from "prop-types";
import "./EmployeeCard.css";

function EmployeeCard({ employeeName, role, department }) {
  return (
    <div className="employee-card">
      <h2 className="employee-name">{employeeName}</h2>
      <p className="employee-role">Role: {role}</p>
      <p className="employee-department">
        Department: {department}
      </p>
    </div>
  );
}

EmployeeCard.propTypes = {
  employeeName: PropTypes.string.isRequired,
  role: PropTypes.string.isRequired,
  department: PropTypes.string.isRequired,
};

export default EmployeeCard;
=====================================================================

2. src/components/EmployeeCard.css
.employee-card {
  background: white;
  border: 2px solid #2980b9;
  border-radius: 10px;
  padding: 20px;
  margin: 12px;
  max-width: 280px;
  display: inline-block;
  vertical-align: top;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
  text-align: center;
}

.employee-name {
  color: #2980b9;
  font-size: 20px;
  margin-bottom: 10px;
}

.employee-role {
  color: #555;
  font-size: 15px;
  margin-bottom: 6px;
}

.employee-department {
  color: #27ae60;
  font-size: 15px;
  font-weight: bold;
}
=====================================================================
```

3. src/App.js

```
import React, { useState } from "react";
import EmployeeCard from "./components/EmployeeCard";
import "./App.css";

function App() {
  const [selectedDepartment, setSelectedDepartment] = useState("All");

  const employees = [
    {
      id: 1,
      employeeName: "Rajan Kumar",
      role: "Software Engineer",
      department: "Engineering",
    },
    {
      id: 2,
      employeeName: "Meena Iyer",
      role: "Product Manager",
      department: "Product",
    },
    {
      id: 3,
      employeeName: "Karthik Raj",
      role: "UI Designer",
      department: "Design",
    },
    {
      id: 4,
      employeeName: "Sneha Pillai",
      role: "QA Engineer",
      department: "Engineering",
    },
    {
      id: 5,
      employeeName: "Divya Nair",
      role: "Data Analyst",
      department: "Product",
    },
  ];

  const filteredEmployees =
    selectedDepartment === "All"
      ? employees
      : employees.filter(
          (employee) =>
            employee.department === selectedDepartment
        );

  const handleDepartmentChange = (event) => {
    setSelectedDepartment(event.target.value);
  };

  return (
    <div id="app-container">
      <h1 id="directory-title">Employee Directory</h1>

      <select
        id="department-filter"
        value={selectedDepartment}
        onChange={handleDepartmentChange}
      >
        <option value="All">All</option>
        <option value="Engineering">Engineering</option>
        <option value="Product">Product</option>
        <option value="Design">Design</option>
```

```
      </select>

      <div id="employee-list">
        {filteredEmployees.map((employee) => (
          <EmployeeCard
            key={employee.id}
            employeeName={employee.employeeName}
            role={employee.role}
            department={employee.department}
          />
        ))}
      </div>
    </div>
  );
}

export default App;
============================================================

4. src/App.css
#app-container {
  max-width: 900px;
  margin: 40px auto;
  padding: 20px;
  font-family: Arial, sans-serif;
}

#directory-title {
  color: #2c3e50;
  font-size: 28px;
  margin-bottom: 16px;
  text-align: center;
}

#department-filter {
  display: block;
  margin: 0 auto 24px auto;
  padding: 8px 16px;
  font-size: 16px;
  border-radius: 6px;
  border: 1px solid #ccc;
}
============================================================

5. src/index.js
import React from "react";
import ReactDOM from "react-dom";
import App from "./App";

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById("root")
);
============================================================

6. src/index.css
body {
  margin: 0;
  padding: 0;
  background: #f0f4f8;
}
============================================================
```

WEEK 11

Code 5 — Recipe Card with Favourite Toggle (React Hooks)

```
2. PAH
1. src/components/RecipeCard.jsx
import React, { useState, useEffect } from "react";
import PropTypes from "prop-types";
import "./RecipeCard.css";

function RecipeCard({ recipeName, cuisine, cookTime }) {
  const [isFavourite, setIsFavourite] = useState(false);

  useEffect(() => {
    console.log("Favourite status changed for:", recipeName);
  }, [isFavourite]);

  const toggleFavourite = () => {
    setIsFavourite(!isFavourite);
  };

  return (
    <div className="recipe-card">
      <h2 className="recipe-name">{recipeName}</h2>

      <p className="recipe-cuisine">Cuisine: {cuisine}</p>

      <p className="recipe-cook-time">Cook Time: {cookTime}</p>

      <button id="favourite-btn" onClick={toggleFavourite}>
        {isFavourite ? "Remove from Favourites" : "Add to Favourites"}
      </button>

      <p id="favourite-status">
        {isFavourite
          ? "Status: Favourite"
          : "Status: Not a Favourite"}
      </p>
    </div>
  );
}

RecipeCard.propTypes = {
  recipeName: PropTypes.string.isRequired,
  cuisine: PropTypes.string.isRequired,
  cookTime: PropTypes.string.isRequired,
};

export default RecipeCard;
=====================================================================

2. src/components/RecipeCard
.css.recipe-card {
  background: white;
  border: 2px solid #c0392b;
  border-radius: 10px;
  padding: 20px;
  margin: 16px auto;
  max-width: 400px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
  text-align: center;
}
```

```
.recipe-name {
  color: #c0392b;
  font-size: 22px;
  margin-bottom: 10px;
}

.recipe-cuisine {
  color: #555;
  font-size: 15px;
  margin-bottom: 6px;
}

.recipe-cook-time {
  color: #888;
  font-size: 15px;
  margin-bottom: 12px;
}

#favourite-btn {
  background: #c0392b;
  color: white;
  border: none;
  padding: 10px 20px;
  border-radius: 6px;
  font-size: 15px;
  cursor: pointer;
  margin-bottom: 10px;
}

#favourite-status {
  color: #27ae60;
  font-size: 15px;
  font-weight: bold;
}
```

==========

3. src/App.js

```
import React from "react";
import RecipeCard from "./components/RecipeCard";
import "./App.css";

function App() {
  const recipes = [
    {
      id: 1,
      recipeName: "Masala Dosa",
      cuisine: "South Indian",
      cookTime: "30 mins",
    },
    {
      id: 2,
      recipeName: "Butter Chicken",
      cuisine: "North Indian",
      cookTime: "45 mins",
    },
    {
      id: 3,
      recipeName: "Pav Bhaji",
      cuisine: "Street Food",
      cookTime: "40 mins",
    },
  ];

  return (
    <div id="app-container">
      <h1 id="page-title">Recipe Book</h1>
```

```
      {recipes.map((recipe) => (
        <RecipeCard
          key={recipe.id}
          recipeName={recipe.recipeName}
          cuisine={recipe.cuisine}
          cookTime={recipe.cookTime}
        />
      ))}
    </div>
  );
}

export default App;
=====================================================================

4. src/App.css
#app-container {
  max-width: 900px;
  margin: 40px auto;
  padding: 20px;
  font-family: Arial, sans-serif;
}

#page-title {
  color: #c0392b;
  font-size: 28px;
  margin-bottom: 24px;
  text-align: center;
}
=======================================================

5. src/index.css
body {
  margin: 0;
  padding: 0;
  background: #f0f4f8;
}
============================================================

6. src/index.js
import React from "react";
import ReactDOM from "react-dom";
import App from "./App";

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById("root")
);
=====================================================================
```

WEEK 12 — PROGRAMS WITH IDENTIFICATION TITLES

Code 1 — Transit Connect Metro Departure Board

```
Week 12
1.COD
src/App.js

import React from 'react';
import './App.css';
import StopCard from './components/StopCard';

function App() {
  return (
    <div className="app">
      <header className="header">
        <h1>Transit Connect</h1>
        <p>Metro Departure Board</p>
      </header>

      <div className="board">
        <StopCard station="Central Square" time={4} />
        <StopCard station="Riverside" time={9} />
        <StopCard station="Airport Link" time={15} />
      </div>
    </div>
  );
}

export default App;

================================================================
src/components/StopCard.jsx

import React from 'react';
import './StopCard.css';

function StopCard({ station, time }) {
  return (
    <div className="stop-card">
      <h2>{station}</h2>
      <p>Due in {time} min</p>
    </div>
  );
}

export default StopCard;

================================================================
src/App.css

body {
  margin: 0;
  background: #11151c;
  font-family: Georgia, 'Times New Roman', serif;
  color: #d8dde5;
}

.app {
  width: 80%;
  max-width: 900px;
  margin: 60px auto;
}
```

```
.header h1 {
  margin: 0;
  color: #ffbd00;
  font-size: 42px;
  letter-spacing: 3px;
}

.header p {
  margin: 8px 0 25px;
  color: #718096;
  font-size: 16px;
  letter-spacing: 2px;
}

.header::after {
  content: '';
  display: block;
  height: 2px;
  background: #a77b00;
  margin-bottom: 20px;
}

.board {
  display: flex;
  flex-direction: column;
  gap: 10px;
}
```

```
================================================================================
src/components/StopCard.css

.stop-card {
  display: flex;
  align-items: center;
  justify-content: space-between;
  background: #1d232d;
  border-left: 6px solid #ffbd00;
  padding: 16px 20px;
  min-height: 42px;
  box-sizing: border-box;
}

.stop-card h2 {
  margin: 0;
  font-size: 19px;
  font-weight: normal;
  letter-spacing: 1px;
}

.stop-card p {
  margin: 0;
  color: #a7c7dc;
  font-size: 15px;
  letter-spacing: 1px;
}

================================================================================
src/index.js

import React from 'react';
import ReactDOM from 'react-dom';
import './index.css';
import App from './App';

ReactDOM.render(
  <React.StrictMode>
```

```
    <App />
  </React.StrictMode>,
  document.getElementById('root')
);
```

```
============================================================
```

WEEK 12

Code 2 — Weather Point Temperature Converter

```
2.COD
src/App.js

import React, { useState } from 'react';
import './App.css';
import CityCard from './components/CityCard';

function App() {
  const [isFahrenheit, setIsFahrenheit] = useState(false);

  const cities = [
    { city: 'Chennai', temperature: 30 },
    { city: 'Delhi', temperature: 25 },
    { city: 'Shimla', temperature: 10 }
  ];

  const toggleUnit = () => {
    setIsFahrenheit(!isFahrenheit);
  };

  return (
    <div className="app">
      <h1>Weather Point</h1>
      <p className="subtitle">City Temperatures</p>

      <button onClick={toggleUnit}>
        {isFahrenheit ? 'Show in Celsius' : 'Show in Fahrenheit'}
      </button>

      <div className="city-row">
        {cities.map((item) => {
          const temperature = isFahrenheit
            ? (item.temperature * 9 / 5) + 32
            : item.temperature;

          return (
            <CityCard
              key={item.city}
              city={item.city}
              temperature={temperature}
              unit={isFahrenheit ? 'F' : 'C'}
            />
          );
        })}
      </div>
    </div>
  );
}

export default App;

================================================================
src/components/CityCard.jsx

import React from 'react';
import './CityCard.css';

function CityCard({ city, temperature, unit }) {
  return (
```

```
    <div className="city-card">
      <h2>{city}</h2>
      <p>{temperature} {unit}</p>
    </div>
  );
}

export default CityCard;

================================================================
src/App.css

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f4f0ff;
  color: #29265c;
}

.app {
  min-height: 100vh;
  text-align: center;
  padding-top: 50px;
  box-sizing: border-box;
}

h1 {
  margin: 0;
  font-size: 30px;
}

.subtitle {
  margin: 8px 0 18px;
  color: #7770a8;
}

button {
  background: #5542ad;
  color: white;
  border: none;
  border-radius: 25px;
  padding: 10px 20px;
  font-size: 14px;
  cursor: pointer;
  margin-bottom: 35px;
}

.city-row {
  display: flex;
  justify-content: center;
  gap: 14px;
}

====================================================================
src/components/CityCard.css

.city-card {
  width: 130px;
  background: white;
  border: 1px solid #e1dcf5;
  border-radius: 10px;
  padding: 18px 10px;
  box-sizing: border-box;
}

.city-card h2 {
```

```
  margin: 0 0 10px;
  font-size: 15px;
  font-weight: normal;
  color: #403b70;
}

.city-card p {
  margin: 0;
  font-size: 20px;
  color: #5542ad;
}
================================================================
```

WEEK 12

Code 3 — NoteCraft Task Board

```
Week 12
CY
src/App.js

import React, { useState } from 'react';
import './App.css';
import TaskForm from './components/TaskForm';
import TaskList from './components/TaskList';

function App() {
  const [tasks, setTasks] = useState([]);

  const addTask = (task) => {
    setTasks((prevTasks) => [...prevTasks, task]);
  };

  const removeTask = (index) => {
    setTasks((prevTasks) => prevTasks.filter((_, i) => i !== index));
  };

  return (
    <div className="app">
      <header>
        <h1>NoteCraft</h1>
        <p>Task Board</p>
      </header>

      <div className="board">
        <TaskForm onAdd={addTask} />
        <TaskList tasks={tasks} onRemove={removeTask} />
      </div>
    </div>
  );
}

export default App;

========================================================================
src/components/TaskForm.jsx

import React, { useState } from 'react';
import './TaskForm.css';

function TaskForm({ onAdd }) {
  const [task, setTask] = useState('');

  const handleAdd = () => {
    if (task.trim() === '') {
      return;
    }

    onAdd(task.trim());
    setTask('');
  };

  return (
    <div className="task-form">
      <input
        type="text"
```

```
        placeholder="Enter task"
        value={task}
        onChange={(e) => setTask(e.target.value)}
      />
      <button onClick={handleAdd}>Add Task</button>
    </div>
  );
}

export default TaskForm;

========================================================================
src/components/TaskList.jsx

import React from 'react';
import './TaskList.css';

function TaskList({ tasks, onRemove }) {
  if (tasks.length === 0) {
    return (
      <div className="task-list">
        <p className="empty-message">No tasks yet</p>
      </div>
    );
  }

  return (
    <div className="task-list">
      {tasks.map((task, index) => (
        <div className="task-row" key={index}>
          <span>{task}</span>
          <button onClick={() => onRemove(index)}>Remove</button>
        </div>
      ))}
    </div>
  );
}

export default TaskList;

============================================================
src/App.css

body {
  margin: 0;
  background: #faf6ed;
  font-family: Arial, sans-serif;
  color: #292929;
}

.app {
  width: 70%;
  max-width: 700px;
  margin: 50px auto;
}

header {
  text-align: center;
  margin-bottom: 25px;
}

header h1 {
  margin: 0;
  font-size: 32px;
}
```

```
header p {
  margin: 8px 0;
  color: #9a8f82;
}

.board {
  background: white;
  border: 1px solid #eadfce;
  border-radius: 8px;
  padding: 16px;
}

================================================================
src/components/TaskForm.css


.task-form {
  display: flex;
  gap: 8px;
  margin-bottom: 20px;
}

.task-form input {
  flex: 1;
  padding: 10px;
  border: 1px solid #e0d5c5;
  border-radius: 6px;
  font-size: 14px;
  box-sizing: border-box;
}

.task-form button {
  background: #bd6328;
  color: white;
  border: none;
  border-radius: 6px;
  padding: 10px 16px;
  cursor: pointer;
  font-size: 14px;
}

================================================================
src/components/TaskList.css

.task-list {
  width: 100%;
}

.empty-message {
  text-align: center;
  color: #aaa096;
  padding: 20px 0;
  margin: 0;
}

.task-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 14px 8px;
  border-bottom: 1px solid #eee5d9;
  font-size: 15px;
}

.task-row:last-child {
  border-bottom: none;
```

```
}

.task-row button {
  background: white;
  color: #bd6328;
  border: 1px solid #d99b75;
  border-radius: 5px;
  padding: 7px 14px;
  cursor: pointer;
}

================================================================================
src/index.js

import React from 'react';
import ReactDOM from 'react-dom';
import './index.css';
import App from './App';

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById('root')
);

================================================================
```

WEEK 12

Code 4 — BrewNova Coffee Menu

1.PAH
src/App.js

```
import React from 'react';
import './App.css';
import MenuItem from './components/MenuItem';

function App() {
  return (
    <div className="app">
      <header>
        <h1>BrewNova</h1>
        <p>Coffee Menu</p>
      </header>

      <div className="menu-board">
        <MenuItem name="Espresso" price={120} />
        <MenuItem name="Cappuccino" price={150} />
        <MenuItem name="Cold Brew" price={180} />
      </div>
    </div>
  );
}

export default App;
```

```
============================================================
```

src/components/MenuItem.jsx

```
import React from 'react';
import './MenuItem.css';

function MenuItem({ name, price }) {
  return (
    <div className="menu-item">
      <h2>{name}</h2>
      <p>Rs. {price}</p>
    </div>
  );
}

export default MenuItem;
```

```
============================================================
```

src/App.css

```
body {
  margin: 0;
  background: #f5efe4;
  color: #3b2b20;
  font-family: Georgia, 'Times New Roman', serif;
}

.app {
  width: 80%;
  max-width: 850px;
  margin: 50px auto;
}
```

```
header {
  text-align: center;
  margin-bottom: 28px;
}

header h1 {
  margin: 0;
  font-size: 36px;
  letter-spacing: 2px;
}

header p {
  margin: 8px 0 18px;
  color: #8b6f5a;
  font-size: 16px;
}

header::after {
  content: '';
  display: block;
  width: 80%;
  height: 2px;
  background: #c7a27b;
  margin: 0 auto;
}

.menu-board {
  background: #fffdf8;
  border: 1px solid #eadbc8;
  border-radius: 8px;
  padding: 18px 22px;
}

============================================================
src/components/MenuItem.css

.menu-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 16px 0;
  border-bottom: 1px dotted #d8c6b2;
}

.menu-item:last-child {
  border-bottom: none;
}

.menu-item h2 {
  margin: 0;
  font-size: 20px;
  font-weight: normal;
}

.menu-item p {
  margin: 0;
  color: #8b4f2c;
  font-size: 15px;
  font-weight: bold;
}

============================================================
src/index.js

import React from 'react';
import ReactDOM from 'react-dom';
```

```
import './index.css';
import App from './App';

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById('root')
);

============================================================
```

WEEK 12

Code 5 — Recipe Search and Filter

2.PAH
src/App.js

```
import React, { useState } from 'react';
import './App.css';
import SearchBar from './components/SearchBar';
import RecipeList from './components/RecipeList';

function App() {
  const [query, setQuery] = useState('');

  const recipes = [
    'Paneer Butter Masala',
    'Veg Biryani',
    'Masala Dosa',
    'Tomato Soup',
    'Paneer Tikka'
  ];

  const filteredRecipes = recipes.filter((recipe) =>
    recipe.toLowerCase().includes(query.toLowerCase())
  );

  return (
    <div className="app">
      <header>
        <h1>Urban Kitchen</h1>
        <p>Recipe Finder</p>
      </header>

      <div className="panel">
        <SearchBar query={query} onSearch={setQuery} />
        <RecipeList recipes={filteredRecipes} />
      </div>
    </div>
  );
}

export default App;

============================================================
src/components/SearchBar.jsx

import React from 'react';
import './SearchBar.css';

function SearchBar({ query, onSearch }) {
  return (
    <div className="search-bar">
      <input
        type="text"
        placeholder="Search recipes"
        value={query}
        onChange={(e) => onSearch(e.target.value)}
      />
    </div>
  );
}
```

```
export default SearchBar;

============================================================
src/components/RecipeList.jsx

import React from 'react';
import './RecipeList.css';

function RecipeList({ recipes }) {
  if (recipes.length === 0) {
    return (
      <div className="recipe-list">
        <p className="empty-message">No recipes found</p>
      </div>
    );
  }

  return (
    <div className="recipe-list">
      {recipes.map((recipe, index) => (
        <div className="recipe-row" key={index}>
          {recipe}
        </div>
      ))}
    </div>
  );
}

export default RecipeList;

============================================================
src/App.css

body {
  margin: 0;
  background: #eef7f1;
  color: #284c3d;
  font-family: Arial, sans-serif;
}

.app {
  width: 70%;
  max-width: 700px;
  margin: 45px auto;
}

header {
  text-align: center;
  margin-bottom: 25px;
}

header h1 {
  margin: 0;
  font-size: 30px;
}

header p {
  margin: 6px 0;
  color: #729887;
  font-size: 14px;
}

.panel {
  background: white;
  border: 1px solid #dfe9e3;
  border-radius: 7px;
```

```
  padding: 14px;
}

============================================================
src/components/SearchBar.css

.search-bar {
  width: 100%;
  box-sizing: border-box;
}

.search-bar input {
  width: 100%;
  padding: 10px;
  box-sizing: border-box;
  border: 1px solid #d9e3dd;
  border-radius: 5px;
  font-size: 14px;
  outline: none;
}

.search-bar input:focus {
  border-color: #284c3d;
}

============================================================
src/components/RecipeList.css

.recipe-list {
  margin-top: 10px;
}

.recipe-row {
  padding: 10px 6px;
  border-bottom: 1px solid #edf1ee;
  color: #365c4c;
  font-size: 14px;
}

.recipe-row:last-child {
  border-bottom: none;
}

.empty-message {
  text-align: center;
  color: #999;
  padding: 20px 0;
  margin: 0;
}

============================================================
src/index.js

import React from 'react';
import ReactDOM from 'react-dom';
import './index.css';
import App from './App';

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById('root')
);

============================================================
```